

第 17 章 嵌入式系统分析与设计

随着智能制造、物联网、人工智能等技术的快速发展,我国嵌入式系统产品需求在不断增加,据统计,我国的嵌入式系统市场规模从 2015 年的 1003 亿元增长到 2020 年的 6446 亿元,是 2015 年的 6 倍多,这么巨大的市场需求潜力,将会带动一大批从事嵌入式系统开发的从业人员。嵌入式系统的快速增长,应用领域也在快速扩张,航空航天、汽车电子、交通、通信、家用电器、电子、金融、网络、监控和工业自动化等领域尤其明显。嵌入式系统是随着微电子技术、软件和传感器技术的发展而快速推进的,但其基本工作原理和特性是有别于我们日常电子系统的。因此,嵌入式系统软件的分析与设计方法也应具备嵌入式系统的特征。

本章在介绍嵌入式系统基础上,从嵌入式数据库系统、嵌入式实时操作系统、嵌入式系统开发方法和系统验证等方面全面介绍了嵌入式系统开发相关知识。

17.1 嵌入式系统概述

嵌入式系统是一种以应用为中心,以计算机技术为基础,可以适应不同应用的功能、可靠性、成本、体积和功耗等方面的要求,集可配置、可裁减的软硬件于一体的专用计算机系统。它具有很强的灵活性,主要由嵌入式硬件平台、相关支撑硬件、嵌入式操作系统、支撑软件和应用软件组成。

1. 嵌入式系统的特点

归纳起来,典型的嵌入式系统具有以下特点:

(1) 专用性强。嵌入式系统是针对具体应用的专门设计的系统,其个性化很强,软件和硬件结合紧密。一般要针对硬件进行软件的开发和移植,根据硬件的变化和增减对软件进行修改。由于嵌入式系统用于完成某一特定任务,整个系统与具体应用有机地结合在一起,其寿命应满足整个系统生命周期。因此,嵌入式产品一旦进入市场,一般具有较长的生命周期。

(2) 实时性强。很多嵌入式系统对外来事件要求在限定的时间内及时做出响应,具有实时性。根据实时性的强弱,通常将嵌入式系统分为实时嵌入式系统和非实时嵌入式系统,其中大部分为实时嵌入式系统。

(3) 软硬件依赖性强。嵌入式系统的专用性决定了其软硬件的互相依赖性很强,两者必须协同设计,以达到共同实现预定功能的目的,并满足性能、成本和可靠性等方面的严格要求。

(4) 专用处理器。嵌入式系统采用的处理器一般是为嵌入式特定目的而专门设计的嵌入式处理器。通常具有可靠性高、功耗低、体积小和集成度高等优点,能够将许多在通用计算机上

需要由板卡完成的任务和功能集成到芯片内部,从而有利于嵌入式系统的小型化和移动能力的增强。

(5) 多种技术紧密结合。嵌入式系统通常是计算机技术、半导体技术、电子技术、机械技术与各行业的具体应用相结合的产物。通用计算机技术也离不开这些技术,但它们相互结合的紧密程度不及嵌入式系统。

(6) 系统透明性。嵌入式系统在形态上与通用计算机系统差异甚大。它的输入设备往往不是常见的鼠标和键盘之类的设备,甚至没有输出装置,用户可能根本感觉不到它所使用的设备中有嵌入式系统的存在,即使知道,也不必关心嵌入式系统的相关情况。

(7) 系统资源受限。嵌入式系统为了达到结构紧凑、高可靠性和低成本的目的,其存储容量、I/O设备的数量和处理器的处理能力都比较有限。

2. 嵌入式系统的组成

嵌入式系统是一种嵌入对象体的结构中或带有执行装置的应用环境中的专用系统,它主要由5个核心功能部件组成,即传感器、处理单元(PE)、作动器、输入接口和输出接口。

(1) 传感器。传感器将物理参数转换为待处理的电信号。例如,热电偶是将热量转换为电信号的热传感器。

(2) 处理单元(PE)。数据处理单元(或嵌入式计算机)接收输入接口提供的数字数据,对数据进行处理,并决定执行某个输出功能。

(3) 作动器。作动器从数据处理单元的输出口接收电信号,并执行物理动作。继电器就是典型的作动器,当继电器被信号激活时,它将打开(或者关闭)某个开关。

(4) 输入接口。实时计算机的输入接口将电信号转换成计算机能够处理的数字形式。传感器有可能嵌入了需要的转换器。

(5) 输出接口。输出接口对实时计算机输出的二进制数据进行处理,将其转换成作动器或其他输出设备需要的信号形式。其基本工作原理如图17-1所示。

嵌入式系统的体系架构主要包括了硬件层、中间层、系统软件层和功能层。

硬件层中包含嵌入式微处理器、存储器(SDRAM、ROM、Flash等)、通用设备接口和I/O接口(A/D、D/A、I/O等)。在一片嵌入式处理器基础上添加电源电路、时钟电路和存储器电路,就构成了一个嵌入式核心控制模块。其中操作系统和应用程序都可以固化在ROM中。

硬件层与软件层之间为中间层,也称为硬件抽象层(Hardware Abstract Layer, HAL)或板级支持包(Board Support Package, BSP),它将系统上层软件与底层硬件分离开来,使系统的底层驱动程序与硬件无关,上层软件开发人员无须

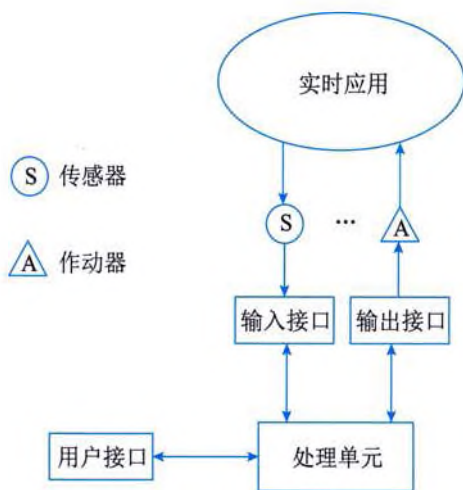


图 17-1 嵌入式系统核心功能部件

关心底层硬件的具体情况,根据 BSP 层提供的接口即可进行开发。该层一般包含相关底层硬件的初始化、数据的输入/输出操作和硬件设备的配置功能。

系统软件层由实时多任务操作系统(Real-Time Operation System, RTOS)、文件系统、图形用户接口(Graphic User Interface, GUI)、嵌入式数据库、网络系统及通用组件模块组成。RTOS 是嵌入式应用软件的基础和开发平台。

功能层主要由实现某种或某几项任务而被开发运行于操作系统上的程序组成。

如图 17-2 给出了一种嵌入式系统的软、硬件组成,其中硬件部分包括嵌入式处理器、存储器和外部设备等硬件基础。软件部分可以分为三类,分别是系统软件、应用支撑软件和应用软件。

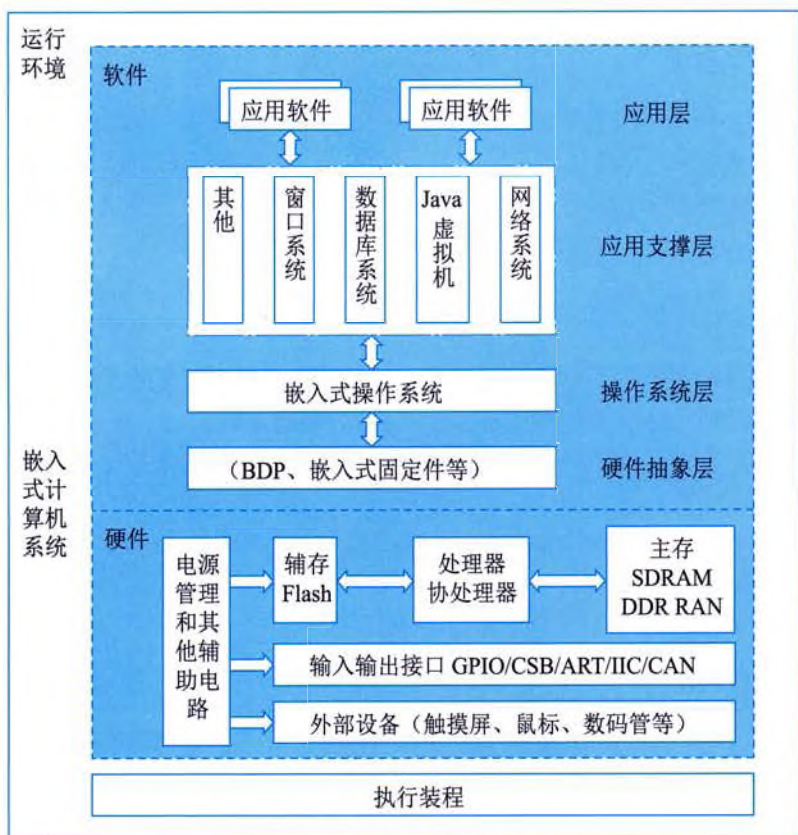


图 17-2 嵌入式计算机的组成

3. 基于多核处理器的嵌入式系统

目前,多核处理器已被广泛应用于嵌入式系统中,其低功耗、多线程并发等优势很好地满足了嵌入式系统需求。那么,在嵌入式系统设计中,如何选择适当的处理器体系结构,是系统设计最为关注的焦点之一。目前,流行的处理器结构包括单核结构、多处理器结构、超线程结构、多核结构、共享 Cache 的多核结构和采用超线程技术的多核结构 6 种,如图 17-3 所示。

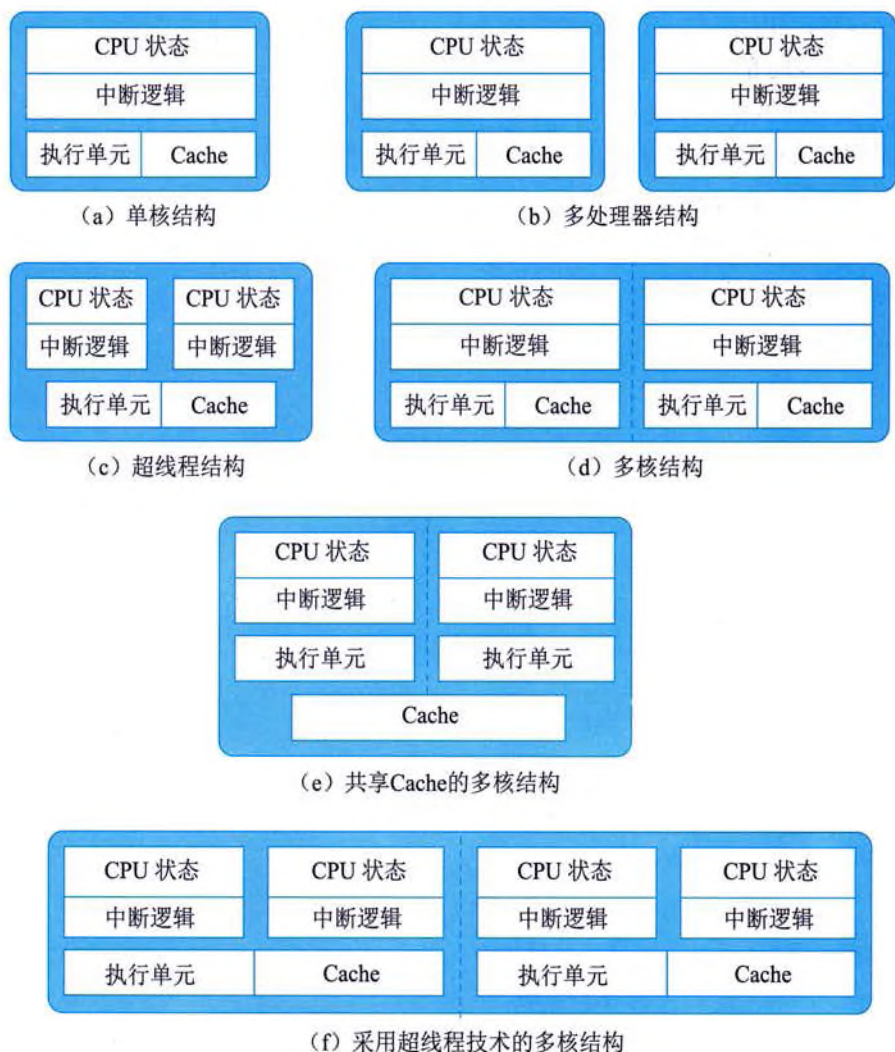


图 17-3 六种处理器体系结构示意图

这 6 种处理器体系结构的主要特点如下：

(1) 单核结构。计算机中仅有一个物理处理器，不支持应用软件的并行执行，因为任何时间点上，CPU 都只能执行一个指令流。

(2) 多处理器结构。支持真正意义上的并行执行，因为多个线程或进程能够在多个处理器上同时执行。

(3) 超线程结构 (SMT)。计算机中只有一个实际的物理处理器，但从软件角度来看，存在多个逻辑处理器，支持操作系统和应用程序将多个线程调度到多个逻辑处理器上，就像多处理器系统一样。从微体系结构的角度看，逻辑处理器的指令是固定的，并且在共享的执行资源上同时执行。

(4) 多核结构。采用单芯片多处理器 (CMP) 的设计，此种结构不是重用单个处理器中某些处理器资源，而是在单个处理器芯片内实现两个或更多的“执行核”。这些“执行核”都是相

互独立的处理器，并具有自己的执行集合以及体系结构资源。

(5) 共享 Cache 的多核结构。与多核结构的工作方式一致，主要差别在于设计时将这些“执行核”设计成可共享片上的 Cache。

(6) 采用超线程技术的多核结构。主要是将多核结构与超线程结构相结合，从而将逻辑处理器的数量增加到“执行核”的两倍。

多核技术的应用，在提高嵌入式系统运算能力之外，可大大降低系统功耗、体积和成本，但在系统确定性方面带来了不利因素。

4. 安全攸关系统

安全攸关系统也称为安全关键系统，是指其不正确的功能或者失效会导致人员伤亡、财产损失等严重后果的计算机系统，比如航空航天领域的飞行控制系统、汽车 ECU 系统等。安全攸关系统是一种安全性、可靠性要求很高的嵌入式系统，为了提高该系统的安全性、可靠性，在系统设计时，需要采用多种措施来提升系统能力。

这里的安全性（Safety）是指单位时间内损失的期望值小于规定的概率。

在嵌入式系统设计中，常用的可靠性、安全性设计模式包括三模块冗余模式（Triple Modular Redundancy Pattern, TMR）和监控 - 启动器模式（Monitor Initiator Pattern）。

TMR 模式是一种用来增强嵌入式系统可靠性和安全性的模式。该模式特别适用于系统没有 fail-safe 状态。其基本原理是提供 3 个并行的运作通道，每个通道之间是相互独立的，通过表决器检查 3 个通道最终的结果，如果结果不一致，采用少数服从多数的原则对结果进行采纳。TMR 模式能够对系统中随机错误进行容错处理。

监控 - 启动器模式适用于具有 fail-safe 状态的嵌入式系统。其基本原理是用一个独立的传感器监控系统是否要进入 fail-safe 状态。如果需要进入，则在系统的启动通道上让系统进入 fail-safe 状态。监控 - 启动器模式解决该问题的方式就是当系统出现错误时，直接将系统转到一个 fail-safe 状态，而不是像 TMR 那样要准备不停运转的三份冗余。监控 - 启动器模式只需要给系统额外增加一个传感器和监控器即可。

当然，在嵌入式系统设计时，系统的完整性（Validity）、可用性（Availability）和可维护性（Maintainability）等指标也是非常重要的考虑因素。

17.2 嵌入式数据库系统

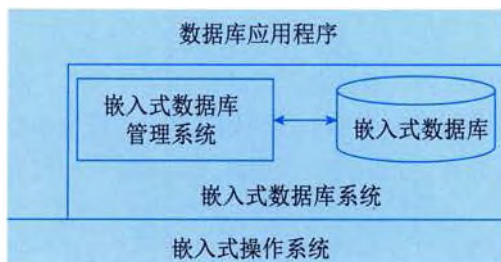


图 17-4 EDBMS 与其他软件的关系

嵌入式数据库管理系统（Embedded Database Management System, EDBMS）是指嵌入式系统中用于存储、管理数据的数据库管理系统。它是一种对资源的要求相对有限的数据库管理系统，它与操作系统及其应用软件集成在一起，固化在嵌入式计算机上，可以支持与嵌入式系统的专门用途密切相关的用途。图 17-4 给出了 EDBMS 与

其他软件的关系。最底层是嵌入式操作系统,如嵌入式 Linux、VxWorks 等,EDBMS 一般需要嵌入式操作系统支持,只有极个别的嵌入式数据库管理系统可以在没有操作系统的条件下运行。

由于用到 EDBMS 的嵌入式系统多是移动信息设备,如掌上电脑、手机、车载设备等,因此,嵌入式数据库也可被称为移动数据库或嵌入式移动数据库。嵌入式数据库是嵌入式系统的重要组成部分,也成为对越来越多的个性化应用开发和管理而采用的一种必不可少有效手段。

与传统数据库相比,嵌入式数据库管理系统应具备以下特点:

(1) 嵌入性。嵌入性是嵌入式数据库的基本特性。嵌入式数据库不仅可以嵌入其他的软件当中,也可以嵌入硬件设备当中。

(2) 实时性。实时性和嵌入性是分不开的。只有具有了实时性的数据库才能够第一时间得到系统的资源,对系统的请求在第一时间做出响应。但是,并不是具有嵌入性就一定具有实时性。要想嵌入式数据库具有很好的实时性,必须做很多额外的工作。

(3) 移动性。移动性是目前在国内提得比较多的一个说法,这和目前国内移动设备的大规模应用有关。可以这么说,具有嵌入性的数据库一定具有比较好的移动性,但是具有比较好的移动性的数据库,不一定具有嵌入性。

(4) 伸缩性。伸缩性在嵌入式场合显得尤为重要。嵌入式场合硬件和软件的平台都是千差万别的,基本都是客户根据需要自己选择的结果。

1. 嵌入式数据库的分类

目前,大多数嵌入式数据库管理系统都是基于关系模型实现的系统。如果以结构模型(关系结构、层次结构、网状结构)为分类原则,则不容易对现有的各种嵌入式数据库管理系统进行区分。因此,在这里采用了一种以嵌入式数据库管理系统的功能侧重点为原则的分类方法。基于这种分类方法可以把嵌入式数据库管理系统分成嵌入式内存数据库管理系统、嵌入式实时数据库管理系统以及移动数据库管理系统三大类。三类数据库管理系统的侧重点如表 17-1 所示。

表 17-1 三类数据库管理系统的侧重点比较

类 型	处理数据对象的性质	功能侧重点
嵌入式内存数据库管理系统	离散、简单的数据	数据库在内存中建立和管理
嵌入式实时数据库管理系统	连续的实时数据流	数据的实时存储和处理
移动数据库管理系统	远程数据	支持移动计算

1) 嵌入式内存数据库管理系统

嵌入式内存数据库管理系统的功能侧重点是解决如何在内存中建立和管理数据库的问题。它所处理的数据对象是离散、简单的数据。在一定程度上,嵌入式内存数据库管理系统可以认为是内存数据库管理系统在嵌入式系统上的实现。可以将它定义为:嵌入式内存数据库管理系统是运行在嵌入式系统上的,可将数据库的主拷贝或工作版本驻留在内存中,依据内存地址对其进行访问和管理的数据库管理系统。数据库的主拷贝或工作版本常驻内存是嵌入式内存数据库的本质特征。

从实际产品的角度看,嵌入式内存数据库管理系统和嵌入式实时数据库管理系统这两个概

念之间有很大的交集。可以说所有的嵌入式实时数据库管理系统都是以嵌入式内存数据库管理系统为基础，但并不是所有的嵌入式实时数据库管理系统都是纯粹的内存数据库管理系统。有很多嵌入式实时数据库管理系统在运行时是采用内外存结合的两级存储模式。嵌入式内存数据库管理系统不一定是嵌入式实时数据库管理系统，主要区别在于对确定的时间限制以及对请求的响应与处理上。

嵌入式内存数据库管理系统适用于需要频繁进行数据存取的嵌入式系统。在这种嵌入式系统中使用嵌入式内存数据库管理系统一方面可以提高应用软件的开发效率，另一方面可以缩短数据的访问时间，提高应用软件的运行效率。典型的嵌入式内存数据库管理系统产品有 FineDB In-Memory、eXtremeDB、SolidDB 等。

2) 嵌入式实时数据库管理系统

嵌入式实时数据库管理系统的功能侧重点是解决如何实时存储和处理数据。它所处理的数据对象是连续的实时数据流。嵌入式实时数据库管理系统是一种可在嵌入式系统上运行的实时数据库管理系统。一方面，它的结构必须很精简，能够适应嵌入式系统内存较小、CPU 处理能力较低的特点；另一方面，它有很强的实时性能，可以保证数据操作请求可以在限定的时间内完成。对于嵌入式实时数据库管理系统来说，数据的正确性不仅依赖于逻辑结果，也依赖于逻辑结果产生的时间。但嵌入式实时数据库管理系统在数据吞吐量、交互方式等方面没有很高的要求。

嵌入式实时数据库管理系统可以认为是一个增强了实时处理能力的内存数据库管理系统。它除了和嵌入式内存数据库管理系统一样要管理物理内存之外，还要在并发控制机制等方面采用专门的技术，以使数据库管理系统能“实时”地响应数据请求。一个嵌入式实时数据库管理系统一般需要的功能包括存储空间管理、数据存取管理、数据安全性控制、事务并发控制、运行日志管理、实时数据转存、数据恢复。其作用是解决大量的、有严格时间要求的数据处理问题，支持这类数据的存储和快速查找。

嵌入式实时数据库管理系统有许多应用系统，如过程控制系统、电网监测系统、智能交通管理系统、网络管理系统等，都需要实时地采集系统运行中的数据并加以处理，以达到对系统进行控制和管理的目的。这些应用系统有以下共同特点：一方面，要维护大量的共享数据；另一方面，数据的采集和处理有很强的时间性，要求在一定的时间内采集数据，存储采集到的数据并处理它们，然后根据处理的结果及时做出响应，否则系统就会失败，甚至带来灾难性的后果。同时，它们所处理的数据往往是“短暂”的，即只在一定的时间范围内有效，超过这个时间范围则毫无意义。对于这样的量很大、时效性很强的数据，仅基于嵌入式操作系统的文件系统已经很难进行存储和管理。因此，产生了对嵌入式实时数据库管理系统的需求。典型的嵌入式内存数据库管理系统产品有 eXtremeDB、Empress、RDM 等。

3) 移动数据库管理系统

移动数据库管理系统的功能侧重点是支持移动计算。它处理的数据对象是远程数据的一个副本。移动数据库管理系统是一种与移动计算环境相适应的数据库管理系统。这种数据库管理系统与操作系统、应用程序集成在一起，在嵌入式系统上运行。它具有通过网络对移动计算环境中其他数据库管理系统的数据操作产生影响的能力。

从数据库技术本身来看,在很多方面移动数据库系统是分布式数据库系统的推广,分布式数据库系统可被认为是移动数据库系统的一种特殊形式。因此,对于分布式数据库管理系统的许多研究成果可以直接应用在移动数据库管理系统中。尽管移动数据库系统和分布式数据库系统两者之间在许多方面是相同的,但是它们之间也存在着不少关键的差别。例如,分布式数据库系统是位置透明的,而移动数据库系统则是位置不透明的。它可以支持与位置相关的各种查询。再如,对分布式数据库管理系统来说,网络断接被认为是一种故障,而对于移动数据库管理系统来说,则认为是一种必须予以处理的正常情况。

移动数据库管理系统的关键技术可以分为两类。一类是与通用数据库相同或基本相同的技术,如存储空间管理、数据存取管理、数据安全性控制、运行日志管理等;另一类是移动数据库系统专用的技术,或者其他类型的数据库也要使用,但为了适应移动数据库系统的需要做了许多改进和调整的技术,如数据库微型化、移动事务处理、数据的缓存与同步、数据广播、移动查询处理等。

移动数据库管理系统的作用主要是解决移动计算环境下的数据管理问题。需要使用移动数据库管理系统的主要是掌上计算机(PDA)、车载设备、智能手机等一些移动信息设备。

目前,已经有很多可供用户选择的移动数据库管理系统。其中有商用软件,也有开源的自由软件,如“小金灵”、OpenBASE Mini、ECOBASE、SQL Anywhere Studio、Oracle Lite、DB2 Everyplace、SQL Server For Windows CE、RDBMS、iMobile Suite、PointBase、Berkeley DB、SQLite等。

2. 使用环境的特点

嵌入式数据库系统是一个包含EDBMS在内的跨越移动通信设备、工作站或台式机,以及数据库服务器的综合系统,系统所具有的这个特点以及该系统的使用环境对EDBMS有着较大的影响,直接影响EDBMS的结构。其使用环境的特点可以简单归纳如下:

(1) 设备随时移动性。嵌入式数据库主要用在移动信息设备上,设备的位置经常随使用者一起移动。

(2) 网络频繁断接。移动设备或移动终端在使用过程中,位置经常发生变化,同时也受到使用方式、电源、无线通信和网络条件等因素的影响。因此,一般并不持续保持网络连接,而是经常主动或被动地间歇性断接和连接。

(3) 网络条件多样化。由于移动信息设备位置的经常变化,导致它们与数据库服务器在不同的时间可能通过不同的网络系统连接,这些网络在带宽、通信代价、网络延迟和QoS等方面可能有所差异。

(4) 通信能力不对称。由于受到移动设备的资源限制,移动设备与服务器之间的网络通信能力是非对称的。移动设备的发送能力都非常有限,使得数据库服务器到移动设备的下行通信带宽和移动设备到数据库服务器之间的上行带宽相差很大。

3. 系统组成

一个完整的EDBMS由若干子系统组成,包括数据库服务器、同步服务器、嵌入式数据库和连接网络等,如图17-5所示。

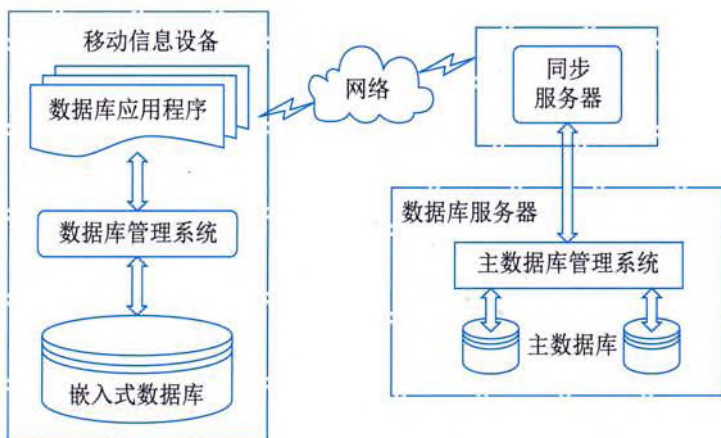


图 17-5 嵌入式数据库系统组成

(1) 嵌入式数据库。嵌入式数据库是一个功能独立的单用户数据库管理系统。它可以独立于同步服务器和主数据库管理系统运行，对嵌入式系统中的数据进行管理，也可以通过同步服务器连接到主服务器上，对主数据库中的数据进行操作，还可以通过多种方式进行数据同步。

(2) 同步服务器。同步服务器是嵌入式数据库和主数据库之间的连接枢纽，保证嵌入式数据库和主数据库中数据的一致性。

(3) 数据库服务器。数据库服务器中的主数据库和 DMBS 可以采用 Oracle 或 Sybase 等大型通用数据库系统。

(4) 连接网络。数据库服务器与同步服务器之间一般通过高带宽、低延迟的固定网络进行连接；根据设备具体情况不同，移动设备与同步服务器之间的连接可以是无线局域网、红外连接、通用串行线或公众网等。

4. “小金灵”移动数据库管理系统

“小金灵”是由人大金仓信息技术公司开发的一款国产移动数据库管理系统。它已经在保险推销、食品卫生监督、炮兵指挥等领域得到广泛应用。“小金灵”移动数据库管理系统被称为 KingBase Lite。它运行于移动信息设备上。从移动数据库应用程序开发者的角度看，KingBase Lite 是一组静态链接库，以这些静态链接库为主构成了一个用于开发数据库的应用程序开发工具包——DTK。通过 KingBase Lite，数据库应用程序可以存取和管理移动信息设备上的数据，或者通过同步服务器访问主数据库。其系统结构如图 17-6 所示。

在“小金灵”移动数据库系统中，同步服务器被称为 SyncServer。它在移动数据库和主数据库之间搭起了一座桥梁。一个 SyncServer 能够同时支持多个移动信息设备同多个数据服务器相连接。移动数据库中的数据是主数据库中某一数据子集的副本，用来保证移动信息设备同数据服务器之间的连接断开时应用程序能正常运行。

KingBase Lite 可以通过串行接口、Internet、无线网等多种方式与 SyncServer 相连接。SyncServer 采用数据同步机制进行数据复制，将移动数据库或主数据库的变化情况传输给对方，以保持数据的松散一致。

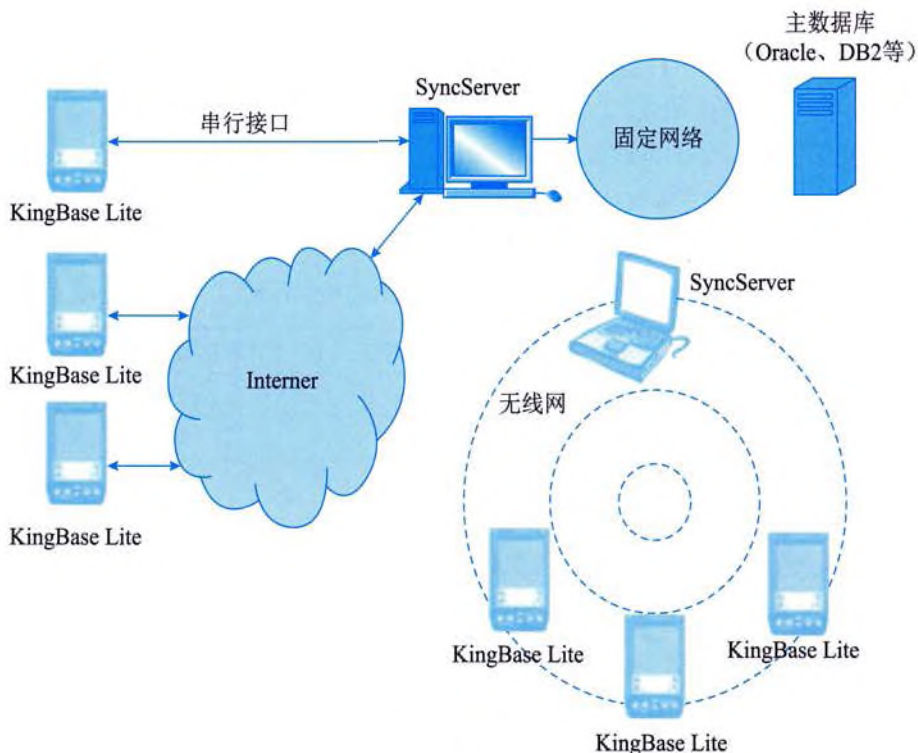


图 17-6 “小金灵”移动数据库管理系统系统结构图

移动数据库应用程序通过 KingBase Lite 的应用程序接口对其进行调用。KingBase Lite 通过 SyncServer 的接口与其相连。SyncServer 则通过 ODBC 访问主数据库管理系统所提供的功能。主数据库管理系统可以采用 Oracle、DB2 等数据库系统。

17.3 嵌入式操作系统

嵌入式操作系统（Embedded Operating System, EOS）是嵌入式系统不可或缺的核心软件。它是一种用途广泛的系统软件，负责嵌入式系统的全部软硬件资源分配、任务调度、控制、协调并行活动等工作。通常包括与硬件相关的底层驱动程序、系统内核、设备驱动接口、通信协议、图形界面、标准化浏览器等。

17.3.1 嵌入式操作系统概述

嵌入式操作系统是指运行在嵌入式系统上，支持嵌入式应用程序的操作系统，是用于控制和管理嵌入式系统中的硬件和软件资源，提供系统服务的软件集合。嵌入式操作系统是嵌入式软件的一个重要组成部分，它提高了嵌入式软件开发的效率和应用软件的可移植性。

1. 嵌入式操作系统的分类

嵌入式操作系统通常根据系统对时间的响应能力分为两类：

(1) 嵌入式硬实时操作系统 (RTOS)。主要面向控制、通信等领域的操作系统。如 WindRive 公司的 VxWorks、ATI 公司的 Nucleus 等，国产的如天脉、瑞华等。

(2) 嵌入式软实时操作系统。主要面向消费电子产品，这类产品包括移动电话、机顶盒、电子书等，操作系统包括 Google 公司的 Android、Apple 公司的 iOS，以及 Microsoft 公司的 WinCE 等，国产的如麒麟、鸿蒙、翼辉等。

二者的特点比较如表 17-2 所示。

表 17-2 硬实时和软实时操作系统的对比

特 征	硬实时操作系统	软实时操作系统（在线系统）
响应时间	要求硬实时	软实时
峰值负载时性能	可预测	降低
节拍控制	环境	计算机
安全性	通常安全关键	非安全关键
数据文件规模	小或中等	大
冗余类型	主动	基于检查点的恢复
数据完整性	短期	长期
错误检测	自动	用户辅助

当然，不同应用领域，对嵌入式操作系统存在不同要求，如果按不同的应用场景分类，有机器人操作系统 (ROS)、智能终端操作系统、车载操作系统、机载操作系统和工业操作系统等。

2. 嵌入式操作系统的特点

与通用操作系统相比，EOS 主要有以下特点：

(1) 微型化。EOS 的运行平台不是通用计算机，而是嵌入式系统。这类系统一般没有大容量的内存，几乎没有外存。因此，EOS 必须做得小巧，以占用尽量少的系统资源。

(2) 代码质量高。在大多数嵌入式应用中，存储空间依然是宝贵的资源，这就要求程序代码的质量要高，代码要尽量精简。

(3) 专业化。嵌入式系统的硬件平台多种多样，处理器的更新速度快，每种处理器都是针对不同应用领域专门设计的。因此，EOS 要有很好的适应性和移植性，还要支持多种开发平台。

(4) 实时性强。嵌入式系统广泛应用于过程控制、数据采集、通信、多媒体信息处理等要求实时响应的场合。因此，实时性成为 EOS 的又一特点。

(5) 可裁减和可配置。应用的多样性要求 EOS 具有较强的适应能力，能够根据应用的特点和具体要求进行灵活配置和合理裁减，以适应微型化和专业化的要求。

3. 一般嵌入式操作系统的结构与组成

与通用计算机系统上的操作系统一样，嵌入式操作系统一般采用层次化架构，以隔离用户与计算机系统的硬件，为用户提供了功能强大的虚拟计算机系统。它通常由硬件抽象层、调试代理、操作系统内核、文件系统和可配置组件（标准库、网络协议栈、设备驱动程序）等功

能组成, 并为应用软件提供标准的 API (Application Programming Interface) 接口服务, 如图 17-7 所示。为方便用户应用程序的开发和代码的复用, 嵌入式系统通常还集成第三方提供的中间件软件, 这些中间件面向特定的应用领域, 具有特定业务逻辑, 具有与平台无关、方便升级和易于移植等特性。

针对不同的硬件平台, 操作系统通常建立在一个抽象硬件层上, 该抽象层位于底层硬件和内核之间, 为内核提供各种方便移植的宏定义接口, 在不同的平台间移植时, 只需要修改宏定义即可。在硬件抽象层中, 封装了与特定硬件有关的各种类型定义、数据结构和各种接口。硬件抽象层提供的接口包括 I/O 接口、中断处理、异常处理、Cache 处理和对称多处理等。根据抽象程度的不同, 硬件抽象层的结构可以分为以下三个级别:

(1) 系统结构抽象层。该层抽象了 CPU 核的特征, 包括中断的传递、异常处理、上下文切换和 CPU 的启动等。

(2) 处理器变种抽象层。该层抽象了 CPU 变种的特征, 如 Cache、内存管理部件、浮点处理器和片上部件 (包括存储器、中断控制器) 等。

(3) 平台抽象层。该层抽象了不同平台的特征, 如片外器件定时器和 I/O 寄存器等。

嵌入式操作系统都有一个内核, 大多数内核都包含任务管理、存储管理、任务间通信、异常和中断管理。从内核结构看, 内核包含调度器、内核对象和内核服务三个公共构件。其中, 调度器是嵌入式操作系统的核心, 提供一组算法决定何时执行哪个任务; 内核对象是特殊的内核构件, 帮助创建嵌入式应用; 内核服务是内核在对象上执行的操作或通用操作。

4. 宏内核和微内核

内核是操作系统的核心部分, 它管理着系统的各种资源。内核可以看成连接应用程序和硬件的一座桥梁, 是直接运行在硬件上的最基础的软件实体。目前, 从内核结构来划分, 可将内核分为宏内核 (Monolithic Kernel)、微内核 (Micro Kernel) 和混合内核。

(1) 宏内核。宏内核管理着用户程序和硬件之间的系统资源, 在宏内核架构中, 用户服务和内核服务在同一空间中实现。也就是说, 内核可以代表内核进程运行代码, 就是通常的内核进程。当用户进程经过系统调用或者中断进入内核态时, 内核也可以代表它运行代码。其优点是实时性高, 用户程序可直接调用内核功能; 结构比较简单, 易于设计。缺点是代码耦合度高, 缺少移植性。

(2) 微内核。内核管理着所有的系统资源, 在微内核中用户服务和内核服务在不同的地址空间中实现。在应用程序和硬件的通信中, 内核仅提供极少的进程和内存管理服务, 而客户端程序和运行在用户空间的服务通过消息的传递来建立通信, 它们之间不会有直接的交互。其优点是结构清晰, 有利于协作开发; 有良好的移植性, 代码量非常少; 有相当好的伸缩性、扩展性。缺点是系统性能偏低。

(3) 混合内核。混合内核的结构其实与微内核相似, 为了提高性能, 混合内核中会将一些



图 17-7 EOS 的一般层次结构

原本微内核中运行在用户层的功能模块放回内核中, 而不常用的, 或者需要的时间很长的功能模块还是放在用户层。

5. 实时性能指标

嵌入式系统的实时性能是由硬件、实时操作系统(Real-Time Operating System, RTOS)和应用程序共同决定的。其中, RTOS 内核的性能起着关键作用。实时嵌入式操作系统和通用操作系统之间的功能有很多相似之处。例如, 它们都支持多任务, 支持软件和硬件的资源管理, 以及都能为应用提供基本的操作系统服务。

在评估实时嵌入式操作系统的设计性能时, 时间是最重要的一个性能指标, 常用的时间性能指标主要有如下几个:

(1) 任务切换时间。任务切换时间也称为上下文切换时间, 是指 CPU 控制权由运行态的任务转移给另外一个就绪任务所需要的时间, 包括在进行任务切换时, 保存和恢复任务上下文所花费的时间, 以及选择下一个待运行任务的调度时间。该指标与微处理器的寄存器数目和系统结构有关。相同的操作系统在不同的微处理器上运行时, 所花费的时间可能不同, 如图 17-8 所示。

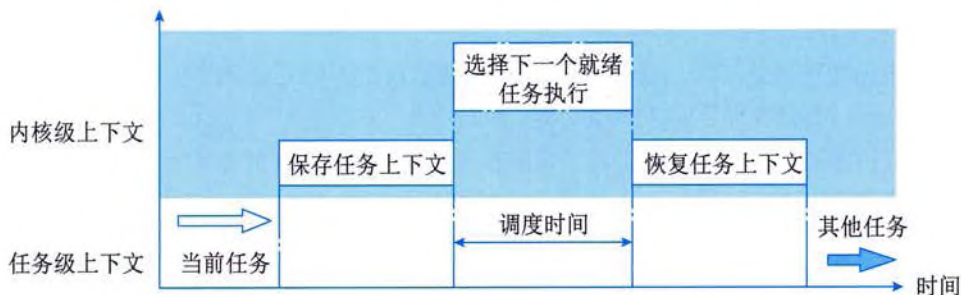


图 17-8 任务切换的时序

(2) 中断处理相关的时间指标。中断时序如图 17-9 所示。

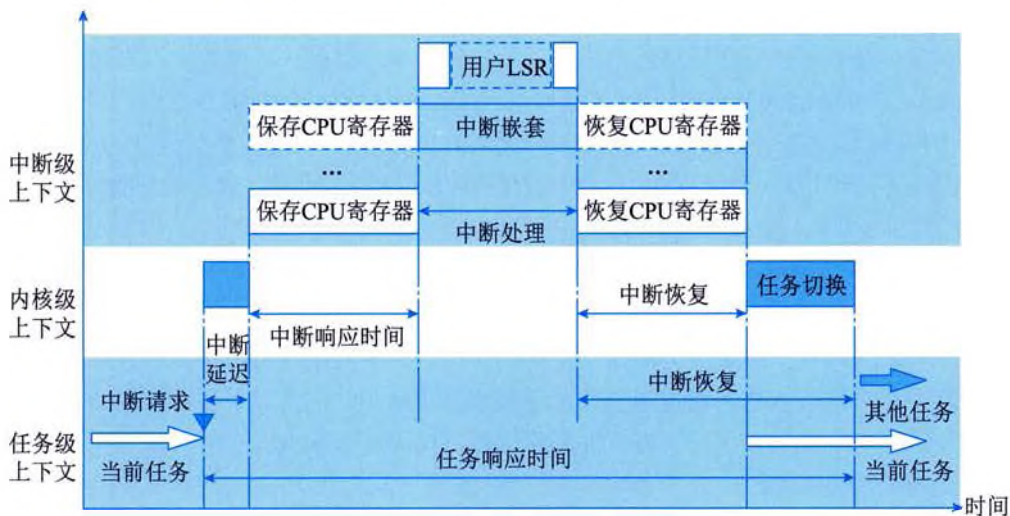


图 17-9 实时内核中断时序

中断延迟时间是指从中断发生到系统获知中断的时间,主要受系统最大关中断时间的影响,关中断的时间越长,中断延迟也就越长。最大关中断时间包含两方面:一是内核最大关中断时间,即内核在执行临界区代码时关闭中断;二是应用关中断时间。关中断最大时间是这两种关中断时间的最大值。

中断响应时间是指从中断发生到开始执行用户中断服务例程的时间;中断恢复时间是指用户中断服务例程结束回到被中断的代码之间的时间。对于可抢占式调度,中断恢复时间还要加上进行任务切换和恢复新的任务上下文的时间;任务响应时间是指从任务对应的中断产生到该任务真正开始运行的时间。

(3) 系统响应时间。系统响应时间指系统在发出处理请求到系统做出应答的时间,即调度延迟,这个时间的大小主要由内核任务调度算法决定。

17.3.2 多任务调度算法

当开发多任务的嵌入式系统应用时,许多普遍性的设计问题随之产生。因为系统的资源是有限的,多个任务执行时,共享和竞争相同的资源不可避免,在可抢占的多任务环境中,资源共享是任务优先级的一个函数,任务的优先级越高,则任务越重要。当访问共享资源时,高优先级的任务先于较低优先级的任务。嵌入式系统中多个任务利用并发执行达到效率最大化,任务之间协同工作也是实际应用的普遍要求。因此,任务之间的通信和同步问题也是设计人员必须考虑的。

1. 调度算法的分类

调度算法根据其时限的性质(软时限还是硬时限)、周期性、可抢占性、静态或动态等准则,可以分为如下几类:

(1) 软时限算法与硬时限算法。实时系统相当复杂,任务优先级的确定与调度并非易事。实时系统大多综合了软实时和硬实时这两种需求,软实时系统只要求任务尽快执行,并不要求在某一特定时间内完成;在硬实时系统中,要求任务不但要正确无误执行,而且还要准时完成。

(2) 周期性与非周期性算法。周期性算法是指调度周期性的任务算法,即每隔 n 个时间单元会执行一次的任务。这 n 个时间单元称为任务周期。非周期性任务是指任务请求处理器的时间是不能预期的。

(3) 可抢占与非抢占算法。可抢占算法是指正在运行的任务可能被其他任务打断,从而放弃 CPU,让其他任务执行;非抢占算法是指任务会一直运行到结束或者等待其他资源而被阻塞,否则不会放弃 CPU。

(4) 静态算法与动态算法。根据任务优先级确定的时机,调度算法分为静态算法和动态算法两类。静态算法是指任务的优先级在设计时就被确定,在任务运行的过程中不会再发生改变。通常,静态调度算法中确定任务优先级的主要依据有执行时间、任务周期和任务的紧迫性。动态算法是指任务的优先级在运行的过程中动态确定,并且会不断地发生变化和更新,该类算法能够完全掌握系统中运行的任务和截止时间、运行时间、优先级,以及到达时间等时间约束,可以灵活地处理变化的系统情况。

(5) 单处理器调度与多处理器调度算法。单处理器算法仅处理一个处理器的情况，多处理器算法可以处理系统中有多个处理器的情况。多处理器算法又分为同质多处理系统和异质多处理系统两种情况。

(6) 在线与离线调度算法。离线调度算法就是运行中使用的调度信息在系统运行之前就被确定，运行的过程中不再变更，离线调度算法具有确定性，但缺少灵活性，适用于那些能够预先知道运行特性，且不易发生变化的应用类型；在线调度算法是指系统运行的调度信息是运行过程中动态收集获取的，如优先级驱动的调度等，该类算法具有最大的灵活性。

当前，大多数内核支持两种普遍的调度算法，即基于优先级的抢占调度和时间轮转调度算法。

- 基于优先级的抢占调度。优先级可以分为静态优先级和动态优先级。应用程序在运行的过程中各任务的优先级固定不变，称为静态优先级。在静态优先级系统中，各任务和它们的时间约束在程序编译时是已知的。应用程序在执行过程中各任务的优先级可以动态改变，称为动态优先级。基于优先级的抢占调度在任何时候运行的任务都是所有就绪任务中具有最高优先级的任务，任务在创建时被赋予了优先级，任务的优先级可以由内核的系统调用动态更改，这使得嵌入式应用对于外部事件的响应更加灵活，从而建立真正的实时响应系统。
- 时间轮转调度算法。为每个任务提供确定份额的CPU执行时间。该调度算法在设计时，应该考虑的因素主要包括CPU的利用率、系统的I/O吞吐量、系统响应时间、公平性和截止时限的满足性等。

2. 单调速率调度算法

对于一个嵌入式实时系统，如果每个任务都能满足时限的要求，则称该系统是可调度的或该系统满足可调度性，也称该系统为健壮的系统（Robust System）。

单调速率调度（Rate Monotonic Scheduling, RMS）算法是一个静态的固定优先级算法，任务的优先级与周期表现为单调函数关系，执行最频繁的任务优先级最高，即任务的周期越短，优先级就越高。RMS是静态调度算法中最有效的算法，如果一组任务能够被任何一种静态调度算法所调度，则在RMS算法下一定是可调度的。RMS算法做了如下假设：

- (1) 所有的任务都是周期性的。
- (2) 任务间不需要同步，没有共享资源，没有任务间的数据交换等问题。
- (3) 系统采用抢占式调度，总是优先级最高且就绪的任务被执行。
- (4) 任务的时限是其下一周期的开始。
- (5) 每个任务具有不随时间变化的定常时间。
- (6) 所有的任务具有同等重要的关键性级别。

要使具有 n 个任务的实时系统中的所有任务都满足硬实时条件，必须使下述的RMS定理成立：

$$\sum_i \frac{E_i}{T_i} \leq n \left(2^{1/n} - 1 \right)$$

式中, E_i 是任务 i 的最长执行时间, T_i 是任务 i 的执行周期。也就是说, E_i/T_i 是任务 i 所需的 CPU 时间。基于 RMS 定理, 要所有的任务满足硬实时条件, 所有具有时间要求的任务总的 CPU 利用时间 (或利用率) 应当小于 70%。通常, 作为实时系统设计的一条原则, CPU 利用率应当在 60% ~ 70% 之间。

任务的可调度性通过计算任务的 CPU 利用率, 然后将该利用率与一个可调度的 CPU 利用率上限进行比较而得到。对于实时系统, 普遍使用的可调度性分析方法是速率单调分析法 (Rate Monotonic Analysis, RMA), 该分析法是基于 RMS 定理提出的。例如, 表 17-3 是一个采样系统使用 RMA 进行分析的结果。

表 17-3 任务的相关参数

周期性任务	执行时间	周 期
任务 1	20	100
任务 2	25	150
任务 3	50	300

根据 RMS 定理, 处理器利用率的计算如下:

$$\begin{aligned}\frac{20}{100} + \frac{25}{150} + \frac{50}{300} &\leq 3(2^{1/3} - 1) \\ \Rightarrow 53.33\% &\leq 77.98\%\end{aligned}$$

该问题的利用率是 53.33%, 低于理论边界 77.98%, 定理的条件满足。因此, 系统是可调度的, 即每个任务都满足时限的要求。

3. 时间轮转调度

在时间轮转调度方式中, 当有两个或两个以上就绪任务具有相同的优先级, 且该优先级是就绪任务最高优先级时, 调度程序会依次调度每个任务运行一个小的时间片, 然后再调度另一个任务。每个任务运行完一个时间片, 不管其是否停止或运行尚未结束, 都要释放 CPU 让下一个任务运行, 即相同优先级的任务会得到平等的执行权利, 释放 CPU 的任务被排到同优先级就绪任务队列的尾部, 等待下次调度。

采用时间轮转调度算法时, 时间片的大小选择至关重要, 会影响系统的性能和效率。如果时间片过大, 时间轮转的调度就失去了意义; 如果时间片过小, 任务切换过于频繁, 处理器的开销大, 用于任务运行的有效时间将降低。因此, 对时间片的灵活调整有助于系统性能和效率的提高, 相同优先级的任务可以具有相同的时间片, 不同优先级类别的任务可以具有不同的时间片。

通常, 纯粹的时间轮转调度无法满足实时系统的要求。取而代之的是基于优先级的抢占式时间轮转调度, 对于优先级相同的任务使用时间片获得相等的 CPU 执行时间。如图 17-10 所示, 任务 1、任务 2 和任务 3 具有相同的优先级, 它们按照各自的时间片运行, 任务 2 被更高优先级的任务 4 抢占, 当任务 4 执行完毕后恢复任务 2 的执行。

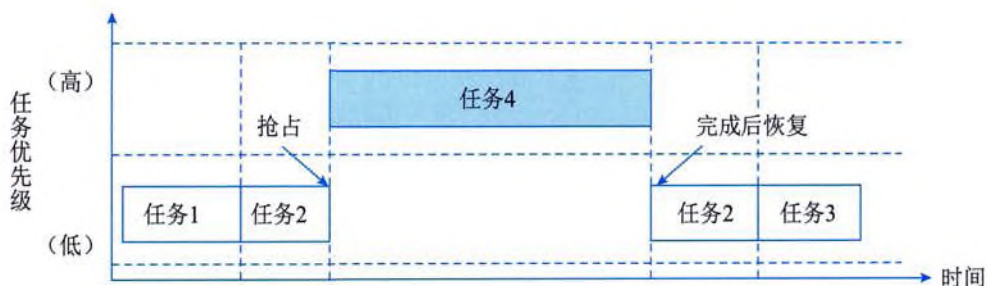


图 17-10 基于优先级的抢占式时间轮转调度

4. 截止时间优先调度

截止时间优先（Earliest Deadline First, EDF）调度算法是指进程的优先级随执行时限变化，即执行时限越靠近，则相对的优先级越高，系统中进程的优先级随时调整。EDF 是一种动态优先级调度算法，根据最大延迟最小化的思想，它基于如下基本定理。

给定一组 n 个独立的任务和一组任意的到达时间，任务可调度性的充分必要条件如下：

$$\sum_i \frac{E_i}{T_i} \leq 1$$

式中， E_i 是任务 i 的最长执行时间， T_i 是任务 i 的执行周期。

EDF 算法要求每个新的就绪任务到达时，都进入就绪任务队列，并根据它们的截止时间排序。如果一个新的已到达任务被插到队列的头部，则当前执行的任务会被抢占。例如，三个任务的抢占调度的情形如图 17-11 所示。

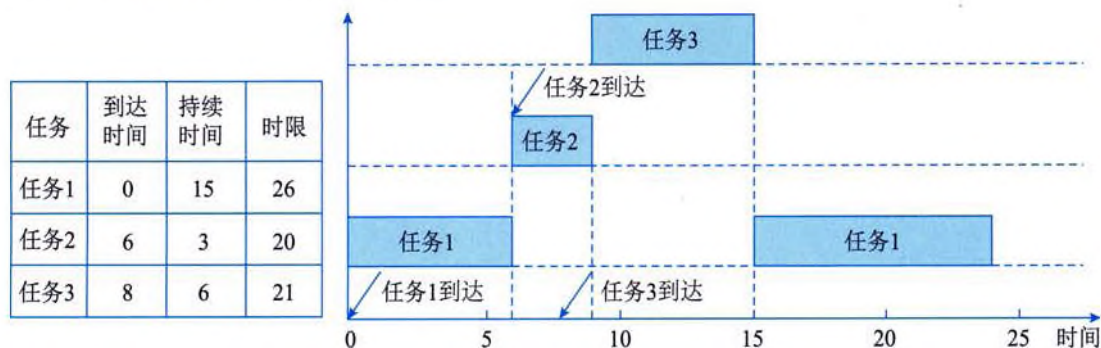


图 17-11 基于截止时间优先调度算法的任务调度图

当任务 1 到达时，它是系统中等待运行的唯一任务，因此立即得到执行。任务 2 在时间 6 到达，由于任务 2 的截止时间（20）早于任务 1 的截止时间（26），因此，任务 2 的优先级比任务 1 的更高，任务 2 抢占任务 1 的 CPU 得到执行。任务 3 在时间 8 到达，由于任务 3 的截止时间晚于任务 2 的截止时间，因此，任务 2 的优先级高于任务 3，任务 2 继续运行，待任务 2 执行结束，任务 3 才能运行。任务 3 执行结束后，任务 1 才能继续运行，在时间 24 执行完毕。

与静态优先级调度算法相比, EDF 算法的优点是其可调度性上限为 100%, 即 CPU 的利用率达到了最高。对于任何一组任务, 如果 EDF 不能满足其调度性要求, 则没有其他算法可以满足这组任务的调度性要求。但在实时系统中, EDF 的实现难度较大, 需要在运行中动态确定任务优先级, 具有较大的调度开销。

5. 基于分区 (partition) 化的分区 APPS 调度算法

分区 APPS (ARINC plus PPS scheduling) 调度算法是一种满足综合化模块化航空电子系统 (IMA) 的嵌入式操作系统调度算法, 是一种静态的任务调度算法, 它将处理器时间资源划分成多个独立时间片资源, 以供多个分区实体可独立享用属于自己的时间片资源。一个计算资源模块内的所有分区都能按事先规定的时间期限完成工作, 则称此计算资源模块所有分区是可调度的。

这里, 分区在时间上具有严格的确定性。分区调度主要完成按固定的、基于周期的时间序列进行 CPU 资源的分配, 每个分区按照主时间框架分配给它的分区窗口 (一个或多个) 被调度程序激活。对分区的特定设置而言, 调度是固定的。

分区调度原则如下:

- (1) 调度单元是分区。
- (2) 分区没有优先级。
- (3) 分区调度算法预先确定, 并按照固定周期重复执行, 在每个周期内至少为每个分区分配一个分区窗口。

分区调度一般采用时间优先调度 (TPS), 即周期小分区被优先安排调度。TPS 必须解决分区时间资源的独立性, 即确保分区时间隔离的有效性。图 17-12 给出了分区调度的时间调度表。

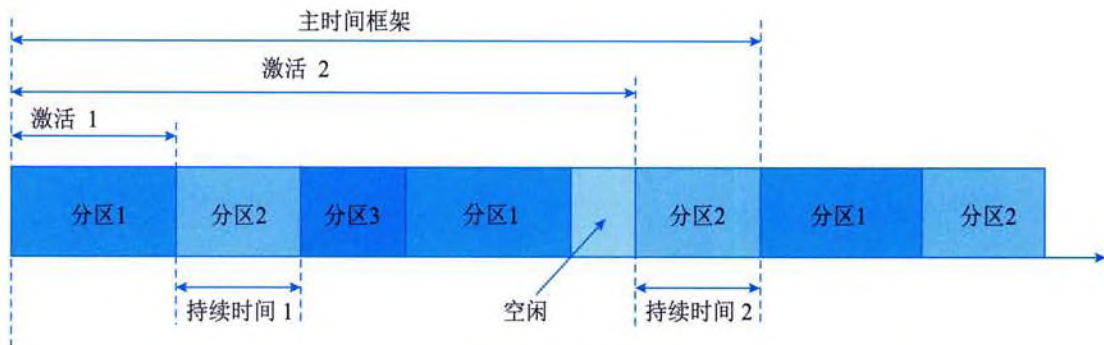


图 17-12 分区调度示意图

分区调度表主要属性包括: Windows ID (时间窗体的标识)、Partition (在时间窗体内被调度的分区标识)、Window offset (针对主时间框架的时间窗体启动时间)、Window Duration (本时间窗体执行的持续时间)。

为了充分利用处理器时间, 当 TPS 有闲置的时间片时, 系统则会变为优先级调度, 这种调度被称为 APPS。图 17-13 给出了 APPS 的调度时序。



图 17-13 APPS 调度时序示意图

17.3.3 优先级反转

优先级反转是指由于资源竞争，低优先级的任务在执行，而高优先级的任务在等待的现象。当具有不同优先级的任务中存在相互依赖关系时，就可能发生优先级反转。例如，如图 17-14 所示，当系统内低优先级的任务 C 占用高优先级任务 A 要使用的资源时，任务 A 只好等待任务 C 执行完毕，并释放该资源后才能被调度执行。这时，如果有中优先级任务 B 进入就绪，剥夺了任务 C 的 CPU 使用权，使得系统只有先让 B 运行完毕，且任务 C 重新运行结束并释放资源后，任务 A 才能运行。

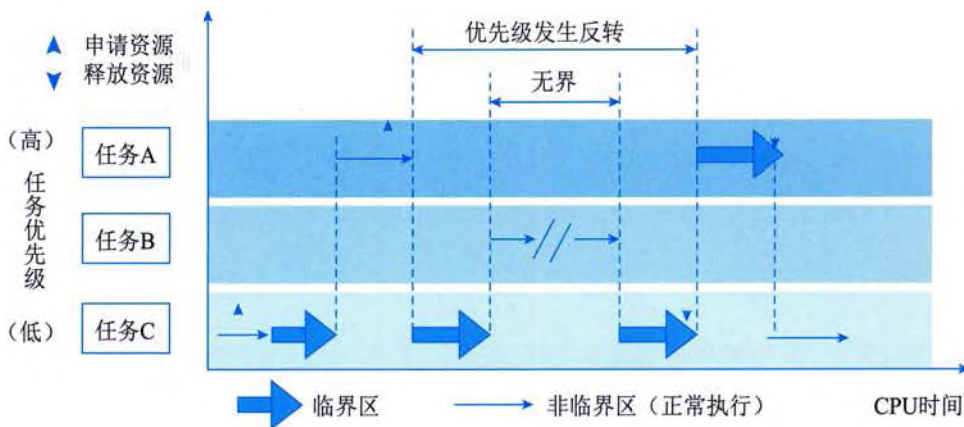


图 17-14 无界优先级反转实例

这样，任务 A 和任务 B 的优先级发生了反转。在这种情况下，高优先级任务的优先级实际上已经降到了低优先级的水平，从而发生优先级反转现象。在图 17-4 中，中优先级的任务抢占低优先级的任务，时间可能不确定，因此称为无界优先级反转。类似地，有界优先级反转的情况如图 17-15 所示。当一个较高优先级的任务请求一个较低优先级任务占有的资源时，较低优先级的任务却锁住了该资源，即使较高优先级的任务就绪，它也必须等待低优先级的任务释放资源后才能继续运行。在这种情况下，低优先级的任务占用资源的时间是已知的，因此称为有界优先级反转。

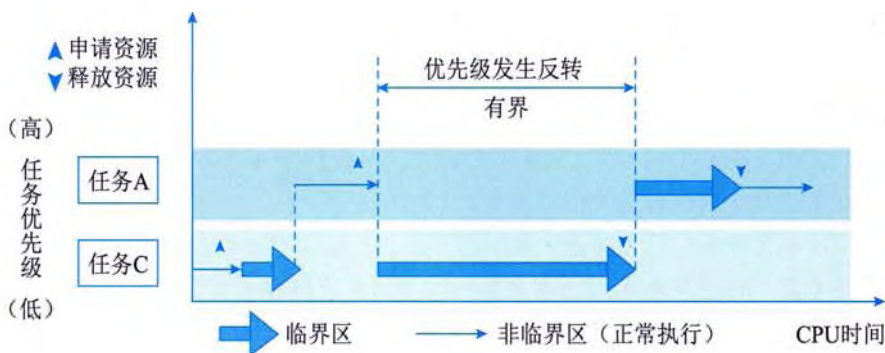


图 17-15 有界优先级反转实例

当优先级发生反转时，某些任务的执行时间减少，同时其他任务的执行时间延长，导致任务错失时限，进而引起时序反常。优先级反转是由不同优先级任务间的资源同步引起的，在实际应用中是不可避免的，但可以使用资源控制协议将其降到最低限度。解决优先级反转问题的常用方法主要有两种：采用优先级继承协议与采用优先级天花板协议。

1. 优先级继承协议

为防止发生优先级的反转，多任务内核应允许动态改变任务的优先级。如果一个任务占有被高优先级任务所请求的资源，那么该任务的优先级会暂时提升到与被该任务阻塞的所有任务中优先级最高的任务同样的优先级水平，当该任务退出临界区时，再恢复到最初的优先级，这种方法称为优先级继承。目前，很多商业内核都具有优先级继承的功能。优先级继承协议规则如表 17-4 所示。

表 17-4 优先级继承协议规则

协议规则号	描述
1	如果资源 R 在使用，则任务 T 被阻塞；如果资源 R 是空闲的，则资源 R 被分配给任务 T
2	当较高优先级的任务 T' 请求资源 R 时，任务 T 的优先级被提升到 T' 的优先级等级
3	当任务 T 释放资源 R 后，返回到先前的优先级

优先级继承的例子如图 17-16 所示。

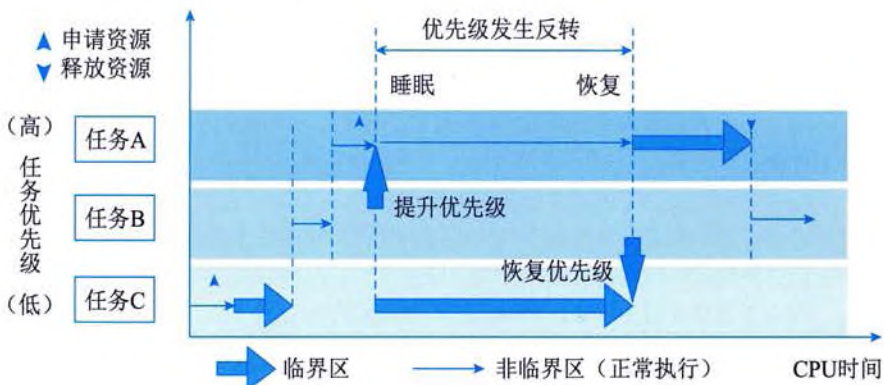


图 17-16 优先级继承的实例

在图 17-6 中，任务 A 是高优先级的，任务 C 是低优先级的。任务 C 首先获得共享资源 S，而任务 A 也请求该资源，优先级继承协议要求任务 C 以任务 A 的优先级执行临界区，这样，任务 C 在执行临界区时，其优先级比它本身的优先级高。这时，中优先级的任务 B 不能抢占任务 C，当任务 C 退出临界区时，又恢复到原来的优先级，使任务 A 仍为最高优先级的任务。这样，任务 A 便不会被中优先级的任务无限期阻塞。

在优先级继承协议中，任务 T 进入临界区而阻塞了更高优先级的任务，则任务 T 将继承被阻塞任务的优先级，直到任务 T 退出临界区。优先级继承协议是动态的，一个不相关的较高优先级任务仍可进行任务抢占，这是基于优先级可抢占调度模式的本性，并且任务优先级在反转期间，被提升优先级的任务的优先级可以继续被提升，即优先级继承具有传递性。虽然在优先级继承协议中，任务的阻塞时间是有界的，但可能出现阻塞链，从而会加长阻塞时间，甚至造成死锁。

2. 优先级天花板协议

优先级继承协议具有死锁和阻塞链问题，而优先级天花板（Priority Ceiling）协议可以解决这些问题。

优先级天花板是指控制访问临界资源的信号量的优先级天花板（简单地说，就是某个临界资源的优先级天花板），信号量的优先级天花板是所有使用该信号量的任务中具有最高优先级的任务的优先级。在任意时刻，一个运行系统的当前优先级天花板是此时所有正在使用的资源中具有最高优先级的优先级天花板。例如，系统中有 3 个资源正在使用，资源 R1 的优先级天花板为 4，资源 R2 的优先级天花板为 6，资源 R3 的优先级天花板为 9，则系统当前的优先级天花板为 9。

在优先级天花板协议下，一个请求任务只可以被一个任务阻塞，不会发生传递阻塞（即产生阻塞链），也不会发生死锁。当一个任务 T 请求资源 R 时，其访问控制协议规则如表 17-5 所示。

表 17-5 优先级天花板协议规则

协议规则号	描 述
1	如果资源 R 在使用，任务 T 不能获得所申请的资源 R，则任务 T 被阻塞
2	如果资源 R 空闲且任务 T 的优先级比当前优先级天花板高，即任务 T 的优先级高于所有当前被其他任务所获取的资源的优先级天花板，则资源 R 分配给任务 T（强迫资源访问次序）；当任务 T 的优先级低于当前优先级天花板时，即使请求的资源 R 空闲时，任务 T 仍会被阻塞，即优先级天花板被阻塞
3	如果当前天花板属于任务 T 当前保持的资源之一，则空闲资源 R 分配给任务 T；反之，任务 T 被阻塞
4	任务 T 将按被分配的优先级执行，除非该任务在临界区执行过程中阻塞了其他高优先级的任务。如果任务 T 阻塞了高优先级的任务，则任务 T 将继承被它阻塞的具有最高优先级任务的优先级，并且按此优先级继续执行，直到它释放每个优先级天花板高于或等于任务 T 的优先级的资源，然后，恢复到原来的优先级

17.4 嵌入式系统开发

嵌入式系统的应用软件是面向特定用户和特定应用的软件系统,是实现嵌入式系统功能的关键。嵌入式系统用途广泛、种类繁多,因此,嵌入式应用软件也多种多样,应用软件的类型和开发难度也千差万别。近年来,随着嵌入式支撑软件的迅速发展,应用软件的开发条件、开发环境和开发效率都得到了极大的改善,同时也使得应用系统的功能不断增强。

17.4.1 开发平台

嵌入式系统的软件开发方法采用的不是通用的开发方法,而是交叉式开发方法,即软件在一个通用的平台上开发,而在另一个嵌入式目标平台上运行。这个用于开发嵌入式软件的通用平台称为宿主机系统,被开发的嵌入式系统称为目标机系统。

1. 交叉开发环境

如图 17-17 所示是一个典型的交叉平台开发环境,包含如下三个高度集成的部分:

- (1) 运行在宿主机和目标机上的强有力的交叉开发工具和实用程序。
- (2) 运行在目标机上的高性能、可裁剪的实时操作系统。
- (3) 连接宿主机和目标机的多种通信方式,如以太网、USB、串口、ICE (In-Circuit Emulator, 在线仿真器) 或 ROM 仿真器等。

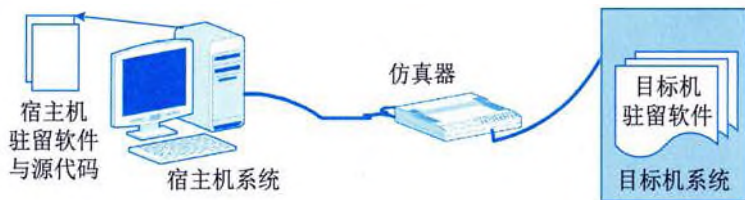


图 17-17 典型交叉平台开发环境

宿主机提供的基本开发工具是交叉编译器、交叉链接器和源代码调试器,作为目标机的嵌入式系统则可能提供动态装载器、链接装载器、监视器和调试代理等。在目标机和宿主机之间有一组连接,通过这组连接,程序代码映像从宿主机下载到目标机,这组连接同时也用来传输宿主机和目标机调试代理之间的信息。

2. 交叉编译环境

如图 17-18 所示是一个典型的交叉编译环境的示例,显示了如何使用开发工具处理各种输入文件,并产生最终目标文件的过程。

在图 17-18 中,交叉编译器将用户编写的 C/C++/Java 源代码文件根据目标机的 CPU 类型,生成包含二进制代码和程序数据的目标文件。在此过程中,交叉编译器会建立一个符号表,包含所产生的目标文件中指向映像地址的符号名,当建立重定位输出时,编译器为每个相关的符号产生地址。

通常,还可以使用归档器将这些目标文件收集到一起,形成一个库。最后,链接器将这些目标文件作为输入来产生一个可执行的映像。一般来说,用户会首先编辑一个链接脚本文

件，用来指示链接器如何组合和重定位各个代码段，以便生成最终文件。此外，链接器还可以将多个目标文件组合成一个更大的重定位目标文件或一个共享目标文件。

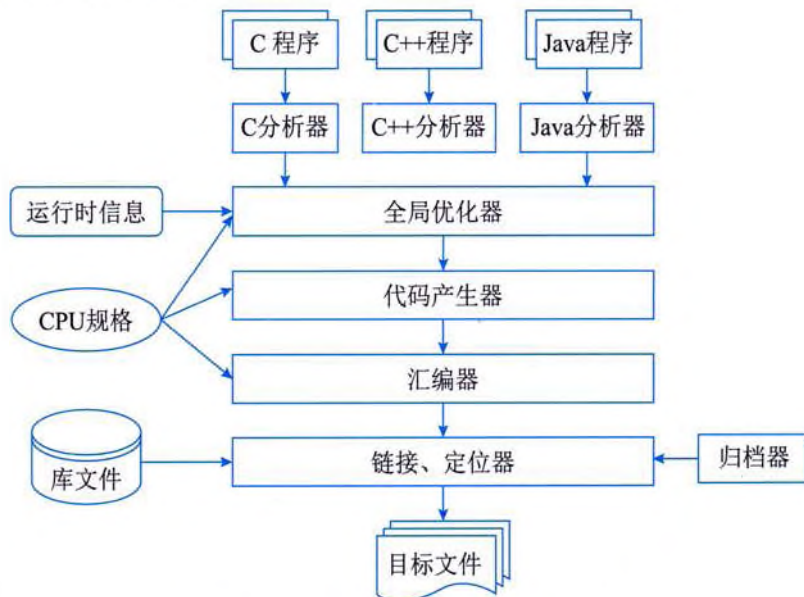


图 17-18 嵌入式系统的交叉编译环境

在嵌入式系统中，一个目标文件通常包含通用信息（如文件尺寸、启动地址、代码段和数据段等具体信息）、机器体系结构特定的二进制指令和数据、符号表和重定位表，以及调试信息等。目前，嵌入式系统中常用的目标文件格式是通用对象文件格式（Common Object File Format, COFF）和可执行链接格式（Executable Linking Format, ELF）。另外，一些系统还需要专门的工具将上述格式转换成二进制代码格式才可使用。

17.4.2 开发流程

嵌入式软件的开发通常与嵌入式硬件紧密相关，其软件需求通常是由系统分配给软件的。系统设计完成后，嵌入式软件开发过程大体可以分为项目计划、需求分析、概要设计、详细设计、编码实现、下载、调试、固化、测试及运行等几个阶段，各个阶段不断反复，直到最终完成设计目标。与通用系统的开发类似，嵌入式软件的开发方法也可以采用软件工程中常见的开发模型，如瀑布模型和螺旋模型等。在开发过程中，设计人员可以根据具体应用的特点和设计目标，选用合适的分析与设计方法。

1. 过程模型

在嵌入式系统开发实践中，形成了一些专门面向嵌入式系统开发的过程模型。例如，嵌入式系统快速面向对象过程（Rapid Object-Oriented Process for Embedded System, ROOPES）是一个迭代的基于UML的过程模型；无线互联网服务工程过程（Wireless Internet Services Engineering Process, WISEP）是一个用于无线互联网服务的专用领域的过程模型。这些模型能够很好地与特定的开发组织和约束条件（例如，应用领域和技术特点等）相适应。

在众多模型中, ROOPES 在实时嵌入式开发环境中得到了大量应用。ROOPES 模型的核心思想是螺旋式迭代。对于大多数嵌入式系统的开发, 完全的瀑布模型和螺旋模型都是不适合的。因为系统开发的初期, 有很长一段时间是开发和生产硬件, 而且生产周期往往比较长, 这就要求硬件的开发要先于软件开发, 且所有的硬件需求在产品运行之前都必须清楚。完全迭代的解决方案在这种情况下是不可能清楚详细需求的。为此, ROOPES 提出了半螺旋式生命周期模型, 如图 17-19 所示, 它将瀑布模型和螺旋模型结合起来, 进行适当裁剪来适应不同的项目。

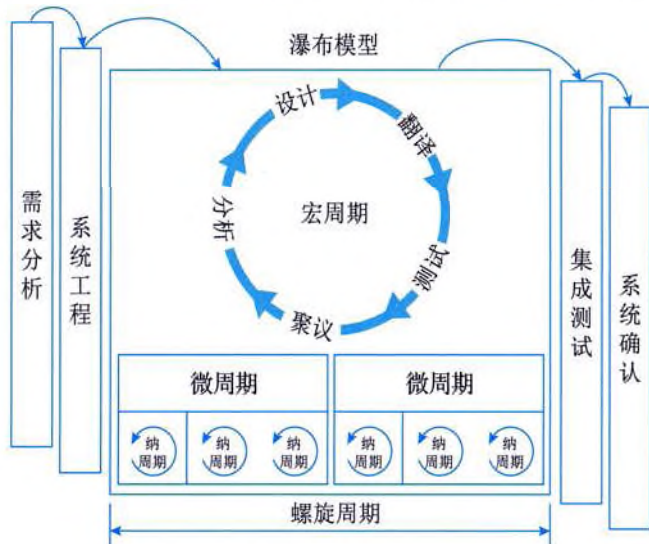


图 17-19 半螺旋过程模型

在半螺旋模型中, 需求分析和系统工程两个阶段在迭代之前进行, 与瀑布模型的生命周期一样, 仅进行一次。但是, 比螺旋模型生命周期中的需求定义要完整得多, 即系统整体的基本功能已经完成。在系统工程阶段, 首先定义完整的高层架构; 然后, 定义架构的子系统, 并进行软件和硬件的划分和功能分配。瀑布模型和半螺旋模型的主要差别在于螺旋部分, 半螺旋模型要创建一系列的原型, 原型的不同部分由不同的小组进行独立开发, 然后在集成和测试阶段汇集到一起。

2. 分析与设计方法

分析与设计方法对于嵌入式系统成功开发具有十分重要的意义, 这是因为良好的设计方法可以使设计人员清楚地了解他们所做工作的进度, 确保不遗漏其中的任何一项工作; 允许使用 CASE 工具帮助设计人员进行工作, 将整个过程分成几个可控的步骤进行。而且, 良好的设计方法方便设计团队成员之间相互交流, 通过定义全面的设计过程, 团队里的每位成员可以很好地理解他们所要做的的工作, 以及完成分配给他们的任务时所达到的目标。

目前, 嵌入式系统常用的分析与设计方法主要有改进的结构化方法、面向对象的方法、基于构件的方法和基于模型的开发方法。

(1) 改进的结构化方法。早期, 实时系统的设计方法主要是结构化设计方法, 在传统的结构化设计方法的基础上, 人们提出了很多针对实时系统的设计方法, 如实时结构化分析与设计

(Real-Time Structured Analysis and Design, RTSAD)、实时系统设计方法 (Design Approach for Real-Time System, DARTS) 和并发实时系统设计方法 (Concurrent DARTS, CODARTS) 等。这些方法本质上是结构化方法和数据流分析方法等传统方法向实时软件开发领域的扩展。采用结构化方法的系统在复用性和可修改性等方面有较大的局限性。

(2) 面向对象的方法。面向对象的实时系统设计方法在可复用性和可修改性等问题上具有明显的优势。采用 OOA 和 OOD 可以很好地对大型、复杂的实时系统进行分析与设计。目前, 已存在不少针对实时系统的面向对象方法。例如, OOD 方法侧重于对象划分和信息隐藏, 但在多任务结构方面的处理能力明显不足; 实时的面向对象建模 (Real-time Object-Oriented Modeling, ROOM) 方法主要着重设计和实现阶段, 为描述对象的行为定义了一套文本和图形语言; 实时系统面向对象技术 (Object-Oriented Technology for Real-Time System, OOTRTS) 方法以 OMT 和融合方法 (Fusion Method) 为基础, 提出了对实时系统响应时间、时间域和并发的处理方法, 并具体提出了对并发、同步、通信、中断处理和硬件界面等方面的处理。OOTRTS 方法将软件开发的主要阶段很好地合并起来, 从规格说明到运行模型之间的过渡紧密自然, 还支持渐进式开发。

(3) 基于构件的方法。构件具有清晰的接口和良好的复用机制, 日益成为提高嵌入式开发效率和改善嵌入式软件质量的一种重要手段。在嵌入式系统设计中, 实时内核、网络协议栈 (TCP/IP)、嵌入式 Web 服务器、嵌入式数据库、嵌入式 CORBA、数据压缩算法和视频处理算法等均可作为开发嵌入式系统的构件。CORBA、EJB、COM/DCOM 等主要的几类构件形式也都会逐步用来构造嵌入式系统。

(4) 基于模型的开发方法 (Mode Based Development, MBD)。基于模型的开发方法主要作用于初期设计阶段, 可建立针对可执行模型的开发和集成框架, 可执行模型可用来描述受控对象和控制计算机系统的行为。嵌入式系统设计采用 MBD 时, 需要选择合适的建模语言以描述模型。常用的建模语言包括 UML、SysML 和 AADL 等。常用的建模工具包括 Rhapsody、SCADE、MATLAB、RTCASE 和 STUDIO 等。

17.4.3 软硬件协同设计

嵌入式系统设计不同于通用系统的软件设计, 通常包含硬件设计和软件设计两方面, 其中前端活动 (如规格说明和系统架构) 需要同时考虑硬件和软件两方面。

1. 软硬件协同设计方法

随着嵌入式系统功能日益强大、复杂度日益提高, 系统设计所涉及的问题越来越多, 难度也越来越大。同时, 软件和硬件时间的界限也变得十分模糊, 不再是截然分开的两个概念, “软件硬化”和“硬件软化”两种趋势同时存在, 软件和硬件的相互影响和相互结合日趋紧密。因此, 出现了软硬件协同设计的方法, 即使用统一的方法和工具对软件和硬件进行描述、综合与验证, 如图 17-20 所示。

协同设计是一种在设计的最初阶段就将软件与硬件两方面结合起来权衡功能的分配, 在软件与硬件的并行设计过程中实现软硬件的交互, 以满足系统的功能与性能要求的设计方法。它

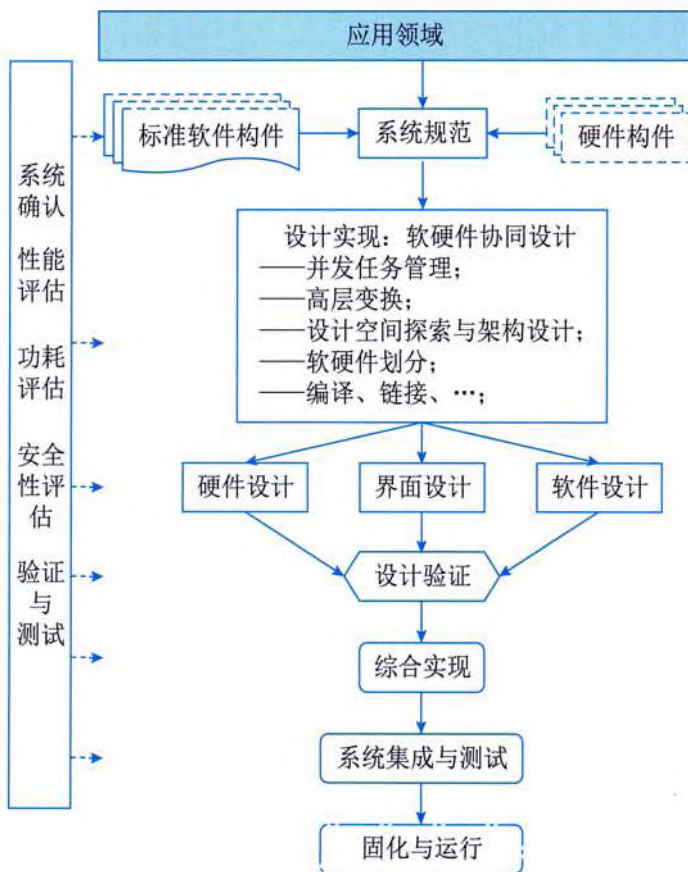


图 17-20 软硬件协同设计方法

的一个最主要的优点就是，设计时充分考虑到现有的软硬件资源情况，在软硬件功能的设计和仿真中，软硬件相互支持，在设计开发的早期阶段，软硬件的互相结合可以及早地发现问题和解决问题，避免了开发后期反复修改带来的一系列问题，有利于降低系统成本，缩短开发周期。

在协同设计过程中，对软硬件的划分要从系统的角度将软硬件完成的功能作均衡，以系统目标为设计标准，在系统的复杂度一定时，使软硬件结合达到更高的性能。软硬件划分好以后，软件和硬件的设计一直是保持并行的，在设计过程中两者交织在一起，互相支持，互相提供开发的平台。软硬件协同的方法可以使软件设计人员在硬件完成之前接触到硬件模块，从而更好地设计硬件的驱动、应用程序和操作系统等软件；同时，可以使硬件设计人员尽早接触软件，为软件设计人员提供高性能的硬件平台，减少设计中的盲目性。

2. 协同设计工具

嵌入式系统的协同设计离不开统一的分析与设计工具的支持。目前，嵌入式系统的协同设计工具可以分为两类：协同合成工具和协同模拟工具。主要的协同合成工具有 POLIS、COSYMA 和 Chinook 等，协同模拟工具有 PTOLEMY 和 TSS 两种。在整个系统设计完成后，在统一框架下模拟不同种类的成分是必要的，协同模拟不仅进行功能检验，而且为用户提供各系统的性能信息，这有助于在系统的早期提出变更方案，不至于造成重大损失。

17.4.4 系统分析与设计

本节以一个多用途通用控制平台为例，对嵌入式系统的开发与设计过程进行描述。该平台是一个面向工业、企事业单位、生活社区和普通家庭用户，集成了移动通信、互联网技术、无线局域网和无线传感器网络等先进的网络技术，以及视频信息采集与处理技术的多用途控制平台。

对于家庭用户来说，可以通过移动通信网络随时随地了解家中状况，可以远程控制家里的电器设备，并且设备出现了故障或紧急情况，该通用控制平台可以主动地通过手机或电话网通知用户或火警等相关部门。特别是出现火灾等极端状况时，控制平台可以启动有关设备进行扑救，同时将相关数据和现场图像或视频资料发送出去，提供给救援人员参考，以便协助救援。对于企业用户，该系统可以设置在仓库、车间和实验室等地点，并且多个平台设备之间可以共享数据和相互协作。

1. 需求分析

嵌入式系统一般都是面向产品开发的，对嵌入式系统进行需求分析是一项复杂而费时的的工作。需求分析阶段最重要的成果之一就是系统规格说明书。对于不同的项目，该文档形式可以灵活变化，但基本包含物理尺寸、操作环境、存放环境、指示灯装置、无线电标准、无线电功率和频率、数据传输率、存储器、平均无故障时间、功耗、电源适配器、散热、系统复位、协议等内容。在进行需求分析时，可以采用 UML 进行需求描述。

2. 系统架构设计

嵌入式系统的架构由硬件架构和软件架构组成，随着嵌入式软件复杂度的日益提高，系统的响应速度和可靠性等重要指标不再仅由硬件架构所决定，软件架构的影响也逐步增大。描述系统如何实现规格说明中定义的功能是系统架构设计的主要目的。但是，在设计嵌入式系统的架构时，很难将软件和硬件完全分开。通常的处理方法是先考虑系统的软件架构，然后再考虑其硬件实现。系统架构的描述必须符合功能上和非功能上的需求，不仅要体现所要求的功能，还要满足成本、速度和功耗等非功能约束。

软件架构的选择和设计在很大程度上取决于嵌入式系统的架构，主要考虑以下几点因素：

(1) 系统的实时性要求。对于硬实时系统，必须进行严格的时序分析。

(2) 采用的设计模型。对于简单的中小型嵌入式系统，可以采用“先硬件后软件”的方法；对于复杂、大型嵌入式系统，必须采用软硬件协同设计的方法，还要充分考虑软件和硬件两方面的互相制约和影响。

(3) 是否需要 EOS。对于复杂的或者日后需要继续开发和移植的系统，最好采用 EOS 来增强系统的可扩展性。

在所有满足系统功能和性能要求的方案中，应该优先选用最简单的架构。

3. 硬件子系统设计

嵌入式系统的开发环境由 4 个部分组成，分别是目标硬件平台、EOS、编程语言和开发工具。其中，处理器和操作系统的选择应当考虑更多的因素，避免错误的决策影响项目的进度。

嵌入式系统设计的主要挑战是如何使互相竞争的设计指标同时达到最佳化。设计人员必须

对各种处理器技术和 IC (Integrated Circuit, 集成电路) 技术的优缺点加以取舍。一般而言, 处理器技术与 IC 技术无关。也就是说, 任何处理器技术都可以使用任何 IC 技术来实现, 但是最终器件的性能、一次性工程 (Non-Recurring Engineering, NRE) 成本、功耗和大小等指标会有很大的差异。通用的可编程技术提供了较大的灵活性, 降低了 NRE 成本, 建立产品样机与上市的时间较快; 定制的技术能够提供较低的功耗、较好的性能、更小的体积和大批量生产时的低成本。根据这些原则, 设计人员便可以对采用的处理器技术和处理器做出合理选择。

一般地, 全定制商用“通用处理器 + 软件”是大多数情况下都适用的一个选择。根据用户的需求和项目的需要, 选择合适的通用嵌入式处理器, 选择时需要考虑处理器的速度、技术指标、开发人员对处理器的熟悉程度、处理器的 I/O 功能是否满足系统的需求、处理器的调试、处理器制造商的支持可信度和封装形式等指标。

在硬件子系统的设计中, 首先, 将硬件划分为部件 (或模块), 并绘制部件连接框图; 其次, 对每个部件进行细化, 将系统分成更多可管理的小块, 可以被单独实现。通常, 系统的某些功能既可用软件实现也可用硬件实现, 没有一个统一的方法指导设计人员决定功能的软硬件分配, 但是可以根据系统约束清单, 在性能和成本之间进行权衡。

嵌入式系统中接口电路的设计需要考虑的因素主要有电平匹配问题、驱动能力和干扰问题。设计时需要注意 I/O 端口、硬件寄存器、内存映射、硬件中断和存储器空间分配等方面。总之, 硬件设计人员应该给软件设计人员更多、更详细的信息, 便于进行软件设计和开发。

4. 软件子系统设计

根据需求分析阶段的规格说明文档, 确定系统计算模型, 然后, 对软件部分进行分析与设计。嵌入式系统的软件设计需要根据应用的实时性要求、应用场合、可用资源情况等诸多约束, 进行合理的任务划分, 这直接影响软件设计的质量。其步骤如下:

(1) 首先, 明确系统的实时性指标。究竟是软实时还是硬实时, 最坏情况下的实时时限是多少等, 以及确定任务大体的数目。任务划分的力度和数目的合理性直接决定着调度开销和任务之间的通信开销, 以及系统响应时间的重要决定因素, 这需要充分考虑极端情况下的各种因素。实时系统的软件应该尽可能简化, 过于复杂的软件设计会增加系统的复杂性, 从而降低可靠性。

(2) 其次, 进行可调度性分析。对已经划分好的任务根据实现的功能特点进行分类, 识别出具有硬实时性要求的所有关键任务, 并给所有的任务赋予适当任务优先级, 根据每个任务执行的时间估计值在考虑调度开销余量的条件下进行粗略的可调度性分析, 从而确保所有关键任务都满足调度时限。根据分析结果再对所有的任务进行适当的分裂与重组。对于周期相同的所有任务的功能进行适当的组合形成一个单独的任务, 以降低事件分发机制的开销; 对于若干固定顺序执行的任务要合成一个单一任务, 避免同步开销; 将若干有相同的事件源触发的任务也合并成一个任务, 以减小事件分发机制的开销; 将计算密集型或以数据处理为主而不进行 I/O 操作的功能独立出来, 由一个优先级的任务来实现; 对任务之间的同步与互斥机制进行分析, 尽量避免大粒度的互斥锁。

(3) 最后, 在具体的实现上, 根据系统特点确定任务的优先级。任务优先级分配的一般原则如下:

- 与中断的关联性。凡是与中断服务程序相关联的任务应该安排尽量高的优先级，以保证及时处理异步事件，提高系统的实时响应能力。否则，CPU可能被其他优先级高的任务长期占用，使得第二次中断发生时，上一次中断还没有处理，从而产生信号丢失，甚至是外部设备失败。
- 紧迫性。越是紧迫的任务对响应时间的要求就越严格，对于所有的紧迫任务，根据它们响应时间的先后顺序，安排优先级。
- 任务的频繁性。对于周期性的任务，执行越频繁，则周期越短，允许耽搁的时间也就越短，故相应的优先级也应该越高，以保证处理的及时性。并且，任务的处理时间越短，优先级也应越高。
- 传递性。信息处理流程的前驱任务的优先级应该高于后继任务的优先级。例如，数据的采集任务优先级应高于数据处理任务的优先级。

5. EOS 的选择

在选择 EOS 时，也需要做如下多方面的考虑：

(1) EOS 的功能。根据项目需要的 EOS 功能来选择 EOS 产品，要考虑系统支持 EOS 的全部功能还是部分功能，是否支持文件系统和人机界面，是实时系统还是分时系统，以及系统是否可裁减等因素。

(2) 配套开发工具。有些 RTOS 只支持该系统供应商的开发工具。也就是说，还必须向 EOS 供应商获取编译器和调试器等；有些 EOS 使用广泛，且有第三方工具可用，因此选择的余地比较大。

(3) EOS 的可移植性。EOS 到硬件的移植是一个重要的问题，是整个系统能否按期完工的关键因素。因此，要选择那些可移植性程度高的 EOS，从而避免 EOS 难以向硬件移植而带来的种种困难，加速系统的开发进度。

(4) EOS 的内存需求。均衡考虑是否需要额外 RAM 或 EEPROM 来迎合 EOS 对内存的较大要求。有些 EOS 对内存的要求是目标相关的，如 Tornado/VxWorks 等，开发人员能按照应用需求分配所需的资源，而不是为 EOS 分配资源。

(5) EOS 附加软件包。EOS 是否包含所需的软件部件，如网络协议栈、文件系统和各种常用外设的驱动等。

(6) EOS 的实时性如何。有些 EOS 只能提供软实时性能，对于需要达到硬实时性能要求的系统就不适用；有些 EOS 既可满足软实时要求，又能满足硬实时要求，如 MS Windows CE 2.0 等。

(7) EOS 的灵活性。EOS 是否具有可剪裁性，即能否根据实际需要进行系统功能的剪裁。有些 EOS 具有较强的可剪裁性，如嵌入式 Linux 和 ECos 等。

6. 编程语言的选择

对于嵌入式系统编程语言的选择，需要考虑通用性、可移植性、执行效率和可维护性等方面。

(1) 通用性。随着微处理器技术的不断发展，其功能越来越专业，种类越来越多，但不同

种类的微处理器都有自己专用的汇编语言。这就给系统开发人员造成了一个巨大的障碍,使系统编程更加困难,软件复用无法实现;而高级语言一般和具体机器的硬件结构联系较少,比较流行的高级语言对多数微处理器都有良好的支持,通用性较好。

(2) 可移植性。由于汇编语言和具体的微处理器密切相关,为某个微处理器设计的程序不能直接移植到另一个不同种类的微处理器上使用,因此可移植性差;高级语言对所有微处理器都是通用的,因此程序可以在不同的微处理器上运行,可移植性较好。这是实现软件复用的基础。

(3) 执行效率。一般来说,越是高级的语言,其编译器和开销就越大,应用程序也就越大、越慢。但单纯依靠低级语言(如汇编语言)来进行应用程序的开发,带来的问题是编程复杂、开发周期长。因此,存在一个开发时间和运行性能之间的权衡。

(4) 可维护性。通常,低级语言程序的可维护性不高。高级语言程序往往是模块化设计,各个模块之间的接口是固定的。因此,当系统出现问题时,可以很快地将问题定位到某个模块内,并尽快得到解决。另外,模块化设计也便于系统功能的扩展和升级。

在嵌入式系统的分析与设计过程中,需要完成一系列的文档,如技术文件目录、技术任务书、技术方案报告、产品规格、技术条件、设计说明书、试验报告和总结报告等,这些文档对完成产品设计和维护相当重要,需要按照通用计算机系统开发的文档管理标准进行管理。

17.4.5 低功耗设计

嵌入式系统通常都是一些移动设备,不能像通用计算机系统那样,长期插接供电电源。因此,在进行嵌入式系统设计时,必须考虑其功耗问题。事实上,低功耗设计是嵌入式系统设计中的难点,是一个系统化的综合问题,必须从软件和硬件两方面全面考虑,才能真正有效地降低功耗。

1. 基于硬件的低功耗设计

对于常用的典型 CMOS 集成电路,其功耗由静态功耗、静态漏电流功耗、内部短路功耗和动态功耗组成。通常,在 CMOS 器件的整体功耗中,动态功耗大约占 70%~90%,静态功耗与静态漏电流功耗一般不大于 2%,内部短路功耗在 10%~30% 之间。从硬件方面降低器件工作时的功耗,主要从降低工作电压方面考虑。对于嵌入式系统设计人员来说,基于硬件的低功耗设计应该从如下几方面考虑:

(1) 板级电路低功耗设计。板级电路的低功耗设计主要围绕处理器的低功耗特性和外围芯片的工作特点。

(2) 选择低功耗处理器。处理器是嵌入式系统不可缺少的核心部件,选择处理器时除了参考其功能、性能、接口和指令集等指标外,还应该考虑功耗特性,在满足正常工作的前提下,尽量选择低电压工作的处理器。

(3) 总线的低功耗设计。对于嵌入式系统来说,总线的宽度越宽,功耗就越大,在可能情况下,应该优先选用低功耗的总线器件,如低电压差分信号器件(Low Voltage Differential Signal, LVDS)。

(4) 接口驱动电路的设计。设计接口驱动电路时,要选用静态电流低的外围芯片,还要考

虑上拉/下拉电阻的选取、悬空引脚的处理和缓冲器使用等。

(5) 分区分时供电技术。嵌入式处理器的工作电压与外围芯片的工作电压通常不一致，可以将它们分别划在不同的供电区域内，并且可以部分关闭空闲分区内的电源，从而降低系统功耗。

2. 基于软件的低功耗设计

在嵌入式系统的设计过程中，通过软件的设计优化，可在一定程度上降低系统的功耗，主要的技术措施有如下几种：

(1) 编译优化技术。对于实现同样的功能，不同实现算法所消耗的时间不同；使用的指令不同，消耗功率也不同。编译器可以通过特定处理器单条指令的功耗基本开销、连续执行不同指令开销的估算模型进行优化。

(2) 软件与硬件的协同设计。可以通过软件与硬件的功能再分配，将用硬件实现的功能由软件实现，从而减少硬件电路的规模，进而达到降低功耗的目的。

(3) 算法优化。对于软件实现的功能，应尽量采用时间复杂度低的算法，如快速傅立叶变换和快速排序算法等，来降低算法执行时间，从而降低功耗。

17.5 嵌入式系统验证

由于嵌入式系统大多数用于航空、航天、轨道交通等安全性、可靠性要求高的领域，其正确性直接关系人身、财产的安全。嵌入式系统开发完成后，如何证明系统正确设计并实现了用户要求，并且是安全、可靠的，这就需要进行嵌入式系统验证。

17.5.1 嵌入式系统的验证与确认

1. 验证和确认的定义

嵌入式系统开发完成后，是否达到预期目的，需要进行验证（Verification）与确认（Validation）工作。验证与确认是一个通用的概念，在标准 EN 50129—2018 中，验证与确认的定义如下：

(1) 验证。通过提供客观证据，确认已满足了规定的要求。

注 1：“验证”用于指定相应的状态。

注 2：设计验证是指应用测试和评估等方法来评估设计是否符合规定的要求。

注 3：在开发的各个阶段进行验证，检查过程和产品，以确定是否符合该阶段开始时规定的要求。

(2) 确认。通过提供客观证据，确认对特定的预期用途或申请的要求已经得到满足。

注 1：“确认”用于指定相应的状态。

注 2：确认的使用条件可以是真实的，也可以是模拟的。

注 3：在设计和开发中，确认涉及检查一个项目以确定是否符合用户需求的过程。

注 4：确认通常在开发的最后阶段，在定义的操作条件下执行，尽管它也可以在早期阶段执行。

注 5: 如果有不同的预期用途, 可以进行多个确认。

从定义看, 验证和确认都是认定。但是, 验证表明的是满足规定要求, 而确认表明的是满足预期用途或应用要求。简而言之, 验证是“正确地做事”, 确认是“做正确的事”。

2. 验证和确认举例说明

以电动汽车的国标 GB/T 39086—2020《电动汽车用电池管理系统功能安全要求及试验方法》为例, 说明验证与确认的分析方法。

本标准对于危害分析、测试等验证确认活动有着清晰的定义, 是验证与确认的一个实践。下面从验证确认思路与系统开发的结合方面论述。

(1) 标准明确定义了系统的安全目标, 包括安全完整性等级 (ASIL)、安全状态和 FTTI (即系统检测出故障到进入安全状态的时间指标), 如表 17-6 所示。

表 17-6 电池管理系统的安全目标

序号	安全目标	ASIL	安全状态	FTTI
1	防止电池单体过充电导致热失控	C	断开高压回路	FTTI 应根据电池系统制造商过充测试结果给出
2	防止电池单体过放电后再充电导致热失控	C	断开充电回路	FTTI 应根据电池系统制造商过放后再充电的测试结果给出
3	防止电池单体过温导致热失控	C	断开高压回路	FTTI 应根据电池系统制造商过温的测试结果给出
4	防止动力蓄电池系统过流导致热失控	C	断开高压回路	FTTI 应根据电池系统制造商过流测试结果给出

注: 充电回路指动力蓄电池从外部吸收能量的回路, 包括外部充电及内部充电 (如整车新动能回收)

(2) 以防止电池单体过温导致热失控为例, 其包括功能的一般要求、运行模式、FTTI 故障容错时间间隔定义、安全状态的进入和退出、告警和降级 5 个方面, 对这个安全功能进行了完整的定义。

(3) 在规定系统的安全功能后, 对每个安全功能的验证方法也进行了规定。如过温保护的验证方法, 验证者根据标准的基本要求, 在模拟环境和真实环境设计各类相关的测试用例, 以验证所要求的安全功能。

针对验证活动, GB/T 39086—2020 标准规定如下。

防止电池单体过温导致热失控为例。

测试目的:

电池管理系统应监测电池单体温度, 当电池单体温度值高于安全阈值时, 使动力蓄电池系统在 FTTI 时间内进入安全状态, 在电池单体过温故障退出、消除条件未满足时, 不应退出安全状态。

测试对象:

测试对象为电池管理系统。

测试要求:

模拟环境下测试应满足如下要求:

①影响测试对象功能并与测试结果相关的所有设备都应处于正常运行状态。

②测试应针对 GB/T 39086 的 7.3.2 小节规定的运行模式。

③模拟电池单体温度信号进行测试。

④模拟的电池单体温度信号，电池单体温度应至少包含低于安全阈值、达到安全阈值及高于安全阈值 3 个温度取值。

⑤测试应对电池管理系统进入安全状态的过程（如安全阈值、时间、状态切换）进行监控。

⑥测试应对电池管理系统退出安全状态的条件进行监控。

真实环境下测试应满足如下要求：

①影响测试对象功能并与测试结果相关的所有设备都应处于正常运行状态。

②测试应针对 GB/T 39086 的 7.3.2 小节规定的运行模式。

③测试对象所在的电池系统应以电池系统制造商许可的充放电倍率进行充放电，或者采用其他电池系统制造商推荐的电池单体加热方法，直到电池单体温度高于安全阈值。

④测试应对电池管理系统进入安全状态的过程（如安全阈值、时间、状态切换）进行监控。

⑤测试应对电池管理系统推出安全状态的条件进行监控。

测试结束条件：

当符合以下任一条件时，结束模拟环境下测试。

①测试对象在故障容错时间间隔内进入安全状态，并无意外退出安全状态。

②测试对象在故障容错时间间隔内进入安全状态，意外退出安全状态。

③测试对象在故障容错时间间隔内未进入安全状态。

当符合以下任一条件时，结束真实环境下测试。

①测试对象在故障容错时间间隔内进入安全状态，并无意外退出安全状态。

②测试对象在故障容错时间间隔内进入安全状态，意外退出安全状态。

③测试对象在故障容错时间间隔内未进入安全状态。

④测试对象所在电池系统发生漏液、泄气、起火或爆炸。

测试通过准则：

测试对象在故障容错时间间隔内进入安全状态，并无意外退出安全状态。

（4）在对安全功能的确认要求中，明确了确认的各个前提条件，对工作条件、失效模式、系统状态处于可监控状态。检查各种失效模式下导致过温后能否在 FTTI 时间内进入安全状态。

针对确认活动，GB/T 39086—2020 标准规定应满足如下要求：

①影响测试对象功能并与确认结果相关的所有设备都应处于正常运行状态。

②确认应在整车层面进行，至少包含真实的电池系统、车辆的实际工况或者模拟车辆使用的工况。

注 1：车辆的实际工况至少包含危害分析和风险评估中最严苛工况。

③确认应包含违背安全目标的典型失效模式。

注 2：典型失效模式包含危害分析和风险评估中导出该安全目标的功能异常，如高温下充电。

④确认应对动力蓄电池系统进入安全状态的过程（如安全阈值、时间和状态切换）进行监控。

- ⑤确认应对动力蓄电池系统的状态进行监控。
- ⑥确认应对动力蓄电池系统退出安全状态的条件进行监控。
- ⑦确认结束后,应在确认环境温度下观察 1 个小时。

从这个标准举例中,对危害分析、安全目标、验证方法、确认方法都进行了定义,实际验证与确认过程远比标准所要求的内容多得多,但从采用的方法上讲,这与验证与确认定义是一致的。

当然,验证与确认还包括其他方面,如工具方法、人员能力、过程符合度。实际的工作中,总是利用有限的资源,开发符合系统规定要求的产品。如何利用有限资源,抓住关键,把验证与确认工作做到实处,确保系统的安全要求得以实现,系统的风险得到了控制。这可不是项目中的验证与确认人员能解决的、需要组织中的系统主管、技术经理等高层人员去解决。

17.5.2 嵌入式系统软件的测试

1. 软件测试概述

通常,软件验证的方法主要包括评审(Review)、分析(Analysis)和测试(Testing)等三种方法。而软件测试是软件开发阶段中非常重要的一个过程,是软件验证工作的核心活动。传统的软件测试是基于过程的测试方法。

软件测试存在多种分类方法,以测试过程中程序执行状态为依据可分为静态测试(Static Testing, ST)和动态测试(Dynamic Testing, DT);以具体实现算法细节和系统内部结构的相关情况为依据可分黑盒测试、白盒测试和灰盒测试三类;从程序执行的方式来分类,可分为人工测试(Manual Testing, MT)和自动化测试(Automatic Testing, AT)。

传统的软件验证与确认活动采用双 V 型的方法,如图 17-21 所示。其中,测试过程主要分为单元测试、集成测试、系统测试和验收测试等 4 个过程。

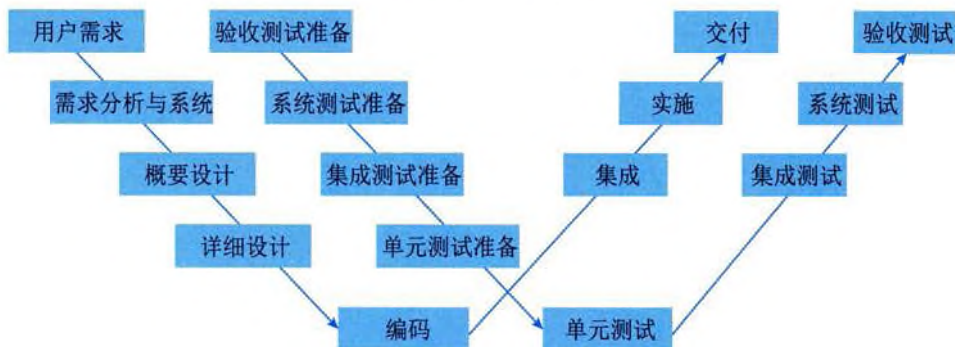


图 17-21 嵌入式软件验证和确认的双 V 型方法

但是,传统测试方法存在以下不可避免的问题:

- (1) 如果测试不够详尽,那么 Bug 就会遗留在产品代码中并潜在地造成严重的问题。
- (2) 测试常常在代码编写完成之后进行。
- (3) 测试人员对软件系统理解不够,会漏掉一些重要内容未被测试到。
- (4) 文档优劣直接影响测试效果。文档和代码往往存在不一致的地方,有可能造成未发现

缺陷。

(5) 如果缺乏自动化测试，不能保证每次重复测试的一致性。

(6) 传统的纠正错误的方法极有可能是在一个地方纠正了错误而又在别的地方引入了新的问题。

2. 基于需求的软件测试方法

民用航空飞行器的机载软件，其安全性是非常重要的，稍有不妥将会引起机毁人亡的重大事故。为了提高机载软件安全性，适航标准中针对机载软件的测试方法制定了详细的标准要求，即 DO-178B/C 《机载系统和设备合格审定中的软件考虑》。DO-178B/C 是美国航空无线电技术委员会为支持含有数字计算机的机载系统和设备的研制工作而制定的软件开发准则，是现行的适航标准。DO-178B/C 标准把软件生命周期分为软件计划过程、软件开发过程和软件综合过程，其中，软件开发过程和软件综合过程又分别被细分为 4 个子过程，如表 17-7 所示。

表 17-7 DO-178B/C 中的软件生命周期过程

软件生命周期过程	包含子过程	DO-178B/C 章节
软件计划过程		第四章
软件开发过程	软件需求过程	第五章
	软件设计过程	
	软件编码过程	
	集成过程	
软件综合过程	软件验证过程	第六章
	软件配置管理过程	第七章
	软件质量保证过程	第八章
	审定联络过程	第九章

其中，软件验证过程规定了机载软件的验证与确认活动。它主要包含对软件产品和软件验证结果进行技术评估，以保证其正确性、合理性、完好性、一致性、无歧义性等特性的所有活动。软件验证依然是一项十分复杂的活动，DO-178B/C 中根据验证活动的分类，分别在 DO-178B/C 标准的附件 A 的表 A-3、A-4、A-5、A-6、A-7 列出了这些验证活动应该实现的目标。

DO-178B/C 提出了基于需求的软件测试方法。将软件测试过程分为三种测试方法：软硬件集成测试、软件集成测试、低级需求的测试。软硬件集成测试是运行在目标机上的，软件集成测试和低级需求的测试环境没有要求。基于需求的软件测试过程见图 17-22 所示。

软件测试可能是一个递进、迭代的过程，通过测试用例的开发可覆盖所有的软件需求，通过运行测试用例可覆盖所有软件代码，并且不断分析，如果有遗漏的需求或代码没有被覆盖，则要增加相应的测试，直到所有需求和代码都被覆盖为止。

基于需求的覆盖率分析（TCA）：分析基于需求测试用例对软件需求的覆盖程度。基于结构的覆盖率分析（SCA）：分析基于需求测试用例对代码结构的覆盖程度。主要包括：①结构覆盖率分析在源代码上进行；②确认结构覆盖率已经达到软件级别要求的程度；③对 A 级软件，如果生成的目标码不能直接追溯到源代码，要进行目标码覆盖率分析（OCA）；④覆盖率标准满足

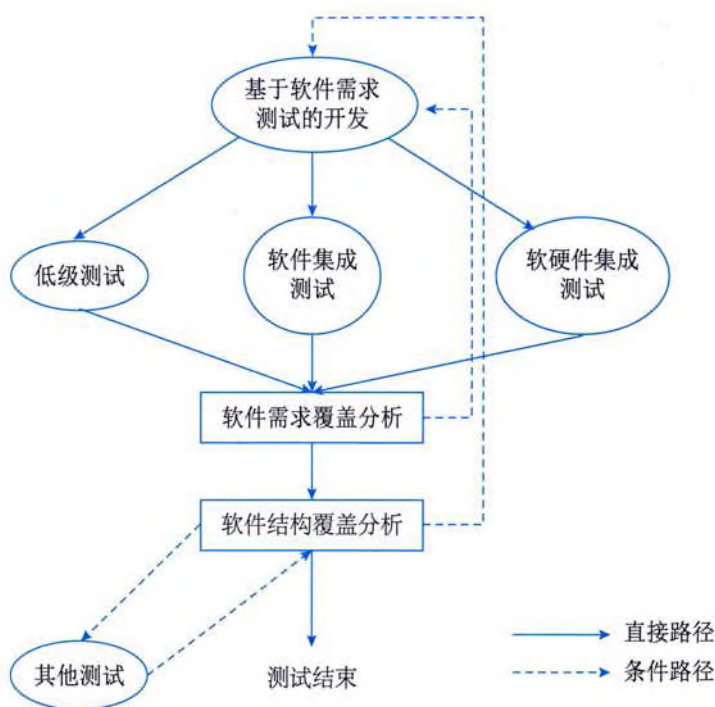


图 17-22 基于需求的软件测试过程

100%TCA 和 SCA。

DO-178B/C 关于不同等级的软件有不同的软件结构覆盖要求，这在 DO-178B/C 表 A-7 验证过程结果的验证中有明确的要求，摘录如表 17-8 所示。

表 17-8 软件级别对应的结构覆盖分析

目 标	软件等级
改进条件 / 判定覆盖 (Modified Condition/Decision Coverage, MCDC)	A
判定覆盖 (Decision Coverage)	A, B
语句覆盖 (Statement Coverage)	A, B, C

语句覆盖是指程序中的每条语句都被调用过至少一次。

判定覆盖是指程序中的每个入口、出口都被调用至少一次，并且程序中每个判定的每个可能结果都至少取到一次。

改进条件 / 判定覆盖是指程序中的每个入口、出口都被调用至少一次，程序中每个条件的每个可能结果都至少被取到一次，程序中每个判定的每个可能结果都至少取到一次。判定中的每个条件都能够被显示地表达，此条件可独立影响判定结果。要让一个条件能独立影响判定结果，可以通过固定其他可能的条件，只更改这个条件来实现。

软件结构覆盖可以在软件源代码或目标码上进行，使用软件覆盖工具进行分析一般需要先对软件代码进行插桩修改，以记录软件运行的路径条件等信息，测试执行结束后将其传输至后台软件进行分析。后台软件分析覆盖数据，得出软件运行的覆盖结果。对这些覆盖结果还要进行人工分析，分析代码结构没有被覆盖的原因。

(1) 如果未覆盖代码有需求而无测试用例，则要增加测试用例。

(2) 如果未覆盖代码无需求，则要增加需求，并设计相应测试用例。注意，增加需求时应该提出问题报告以记录需求的缺失。

(3) 如果是死代码，即因为设计错误而永远无法被执行或使用的代码，则要提出问题报告来删除。

(4) 如果是未激活代码，即不准备被运行或使用的代码，或在特定硬件的配置条件下才会被执行的代码。则要给出分析描述这些代码是正确的，并且这些代码不会被错误地执行到。

根据 A 级软件要求，软件测试要满足 100%MCDC 覆盖的要求。100% 覆盖即所有软件代码都要被测试所运行到，未运行到的代码也用上面所说的方法进行分析，并给出合理的分析结果。

覆盖分析的工作可以在测试程序基本开发运行完成后，即测试的后期才执行。这时运行覆盖可以得到较高的覆盖率，相对节省时间。另外，也可以和一般的测试同步进行，即从开始就在两种代码上运行，这时检查代码的覆盖情况可以看出代码是否按预期运行，这对保证代码的正确执行有好处，当然这种方法花费的人力和时间成本也是巨大的。这两种方法有利有弊，而 DO-178B/C 并没有规定覆盖应该在哪个阶段开始进行，可以根据自己的需要选择，当然最终覆盖结果是要求基于最终版代码的。

3. 形式化方法

形式化方法的本质是基于数学的方法来描述目标软件系统属性的一种技术。不同的形式化方法的数学基础是不同的，有的以集合论和一阶谓词演算为基础（如 Z 和 VDM），有的则以时态逻辑为基础。

形式化方法需要形式化规约说明语言的支持。形式化方法的一个重要研究内容是形式规约（Formal Specification，也称形式规范或形式化描述），它是对程序“做什么”（what to do）的数学描述，是用具有精确语义的形式语言书写的程序功能描述，它是设计和编制程序的出发点，也是验证程序是否正确的依据。

形式化方法的意义在于能帮助发现其他方法不容易发现的系统描述不一致、不明确或不完整的情况，有助于增加软件开发人员对系统的理解，因此形式化方法是提高软件系统，特别是安全攸关系统的安全性、与可靠性的重要手段。

针对不同的系统，需要采用不同的形式化方法，每种形式化方法都有不同的数学定义，可通过类型分析决定应用程序采用何种形式化方法。通常，形式化方法分为如下几类：

(1) 面向对象的形式化方法，通过定义状态和操作进行建模，如 Z 语言、VDM、B、Object-Z 等方法。

(2) 面向属性的形式化方法，如 OBJ3、Larch 等方法。

(3) 基于并发性的形式化方法，如 CCS、ACP、CSP、LOTOS 等方法。

(4) 基于实时性的形式化方法，如 TRIO、RTOZ 等方法。

如果根据描述方式，形式化方法还可划分为：模型描述的形式化方法，即模型描述的形式化方法通过构造一个数学模型来直接描述系统或程序；性质描述的形式化方法，即性质描述的形式化方法通过对目标软件系统中不同性质的描述来间接描述系统或程序。

- 使用白盒测试确认构件的内部逻辑、数据和程序结构。
- 使用黑盒测试确认构件的外部功能、行为和接口。
- 度量其性能。
- 报告测试结果、程序错误和测试覆盖率。

(2) 集成测试是发现构件与应用软件交互的错误，其任务是：

- 构件之间的通信和交互关系。
- 需要关注的两个问题是构件的集成顺序和如何测试新集成的构件。
- 选择合适的构件，就需要考虑构件是否符合系统需求、构件质量是否足够好、构件对系统其他部分是否有影响。

(3) 系统测试是测试整个系统的功能与性能。

(4) 回归测试的主要任务是：

- 确定变化。构件规约、接口、设计、程序代码。
- 分析变化影响。构件功能、接口、结构、行为。
- 选择可复用的测试用例。
- 增加新的测试用例。

当然，基于系统的构件测试才刚刚起步，目前还没有比较实用的构件测试准则，仍然沿用传统软件测试的某些准则，如白盒测试、黑盒测试等，测试人员可根据实际情况提出自己的准则。